Treaps

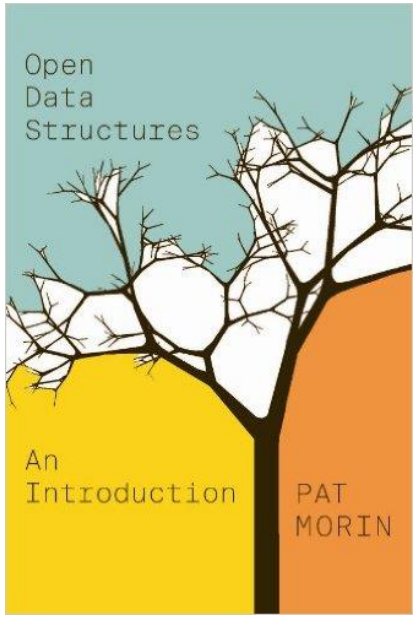
COMP 2402

Abstract Data Types & Algorithms

Readings

Today's topic

- Treaps
  - Chapter 7.2



Sorted Sets (SSet)

	find, add, remove
Skip Lists	expected $O(\log n)$
BST	$O(height) \in O(n)$
Random BST	expected $O(\log n)$ [find]
Treap	expected $O(\log n)$

Binary Search Trees

In a **binary search tree**, each node stores some data from some **total order**

The **binary search tree property**: (with distinct items)

For any node u

1. All data values stored in the subtree u.left are less than u.x
2. All data values stored in the subtree ru.right are greater then u.x

Random Binary Search Trees

Theorem 7.1: A random binary search tree can be constructed in time $O(n \ln(n))$ time. In a random binary search tree, the `find(x)` operation takes $O(\ln(n))$ expected time.

Note: The expected runtimes of the random BST are based on the random permutation used to generate the BST. It does not include adding and removing elements after this tree is created.

TREAP

A **treap** is a binary tree data structure that obeys both the **binary search tree property** and the **heap property**.

Each node has two pieces of information

- x is the data
- p is the priority (which is assigned randomly)

TREAP

Each node has two pieces of information

- x is the data

Binary search tree property: For node u, all data in the subtree u.left is less than u.x and all data in the substree u.right is greater than u.x

TREAP

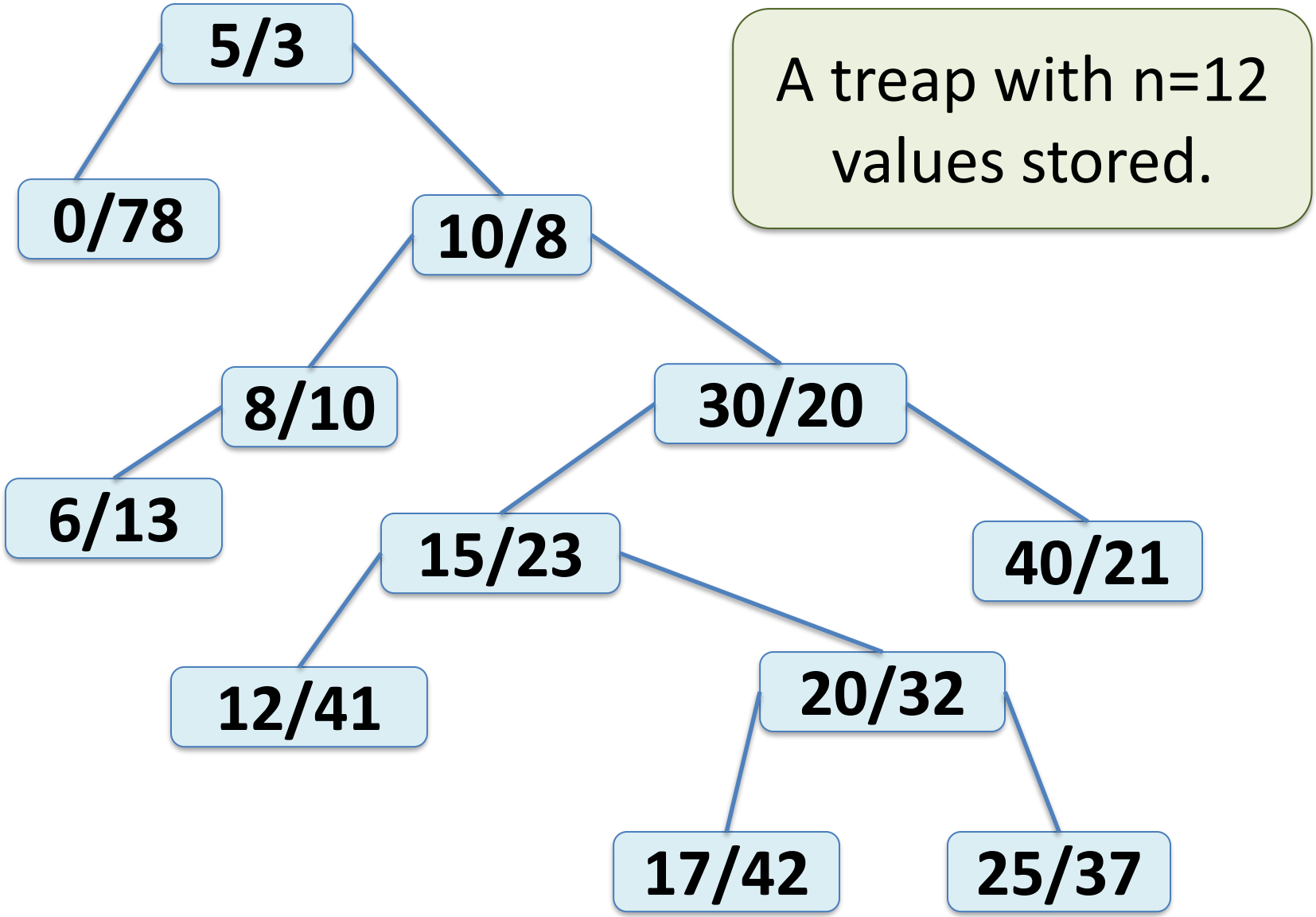
Each node has two pieces of information

- p is the priority (which is assigned randomly)

Heap property: at every node u, except the root

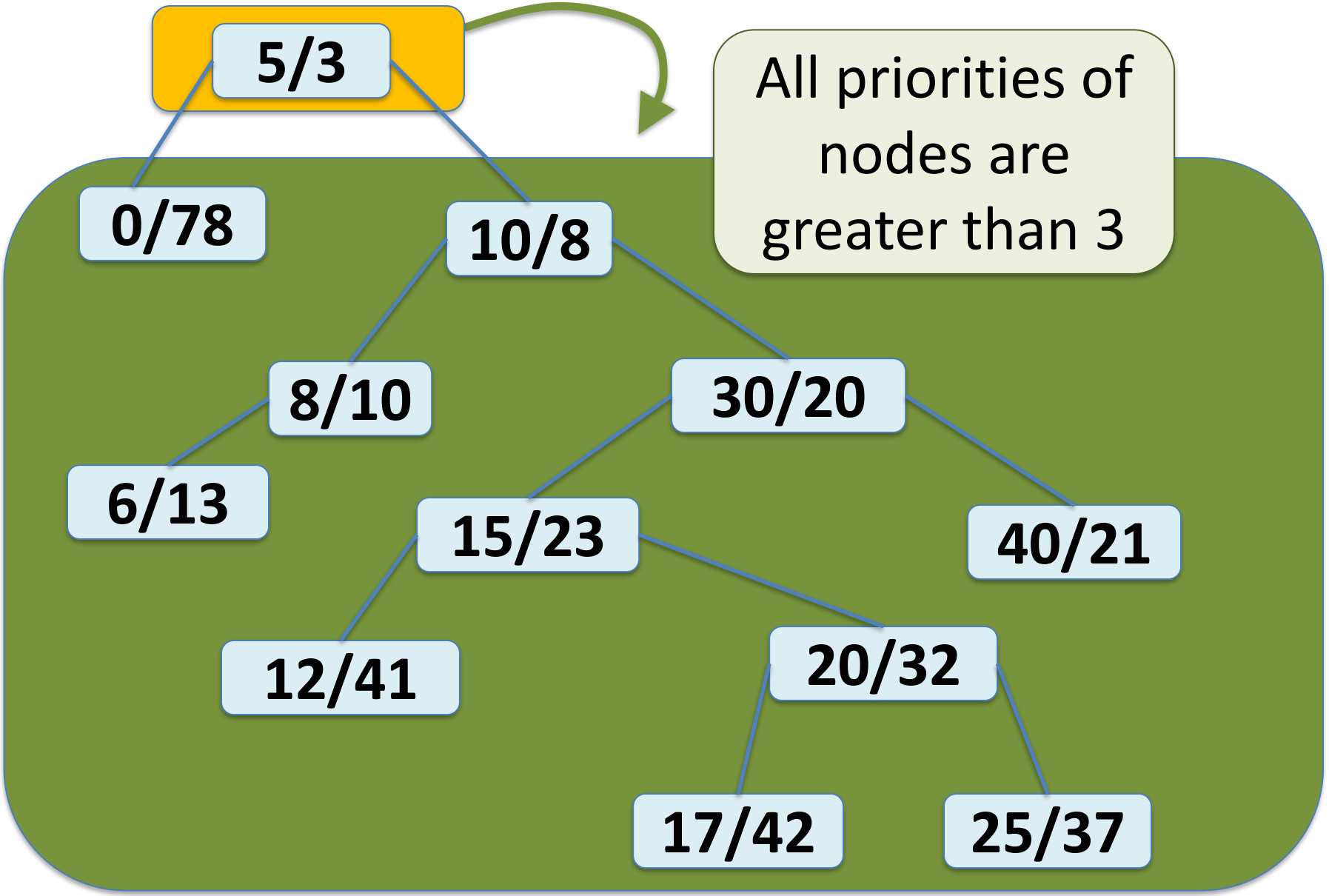
$$u.parent.p < u.p$$

TREAP

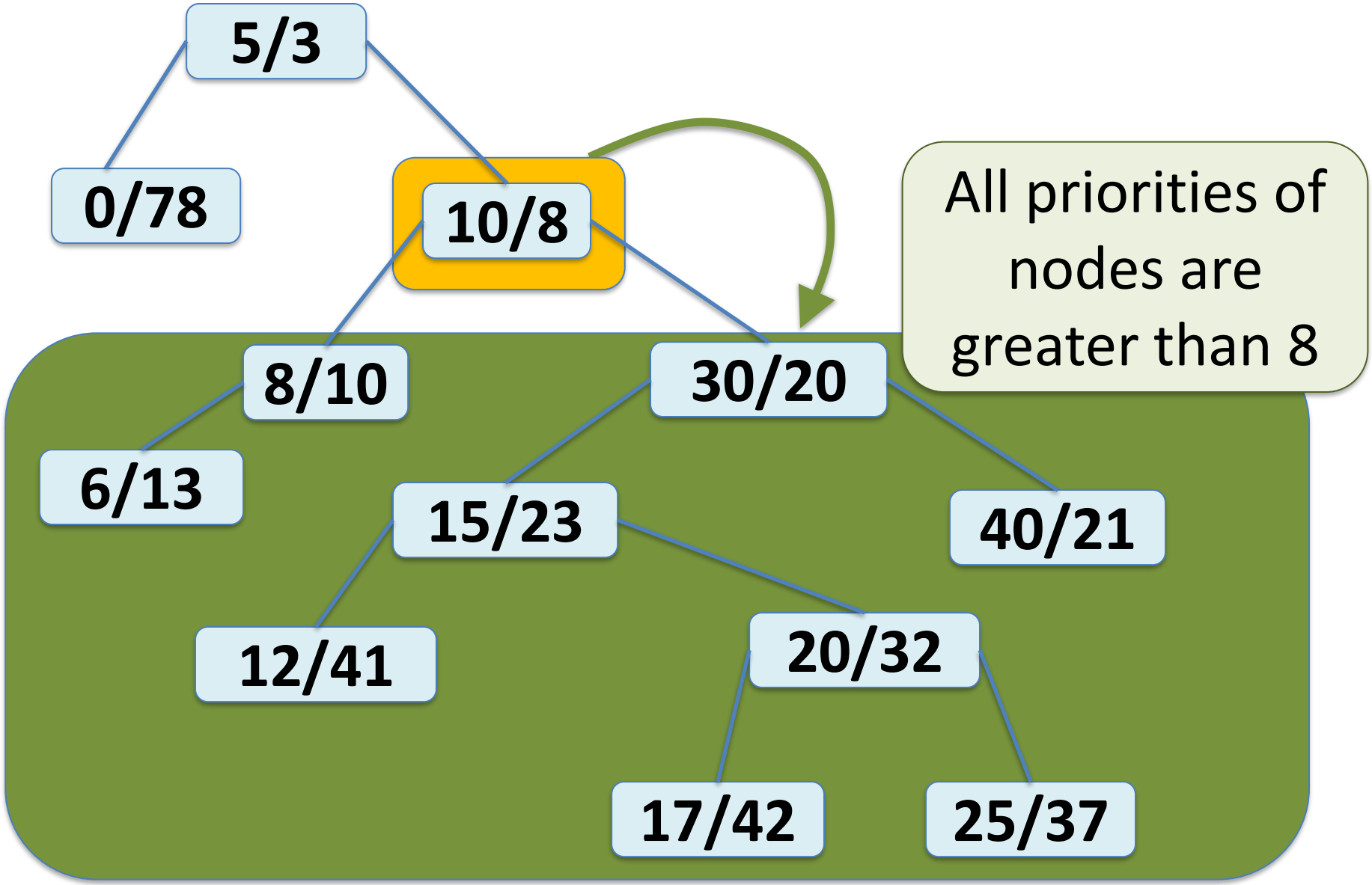


A treap with n=12 values stored.

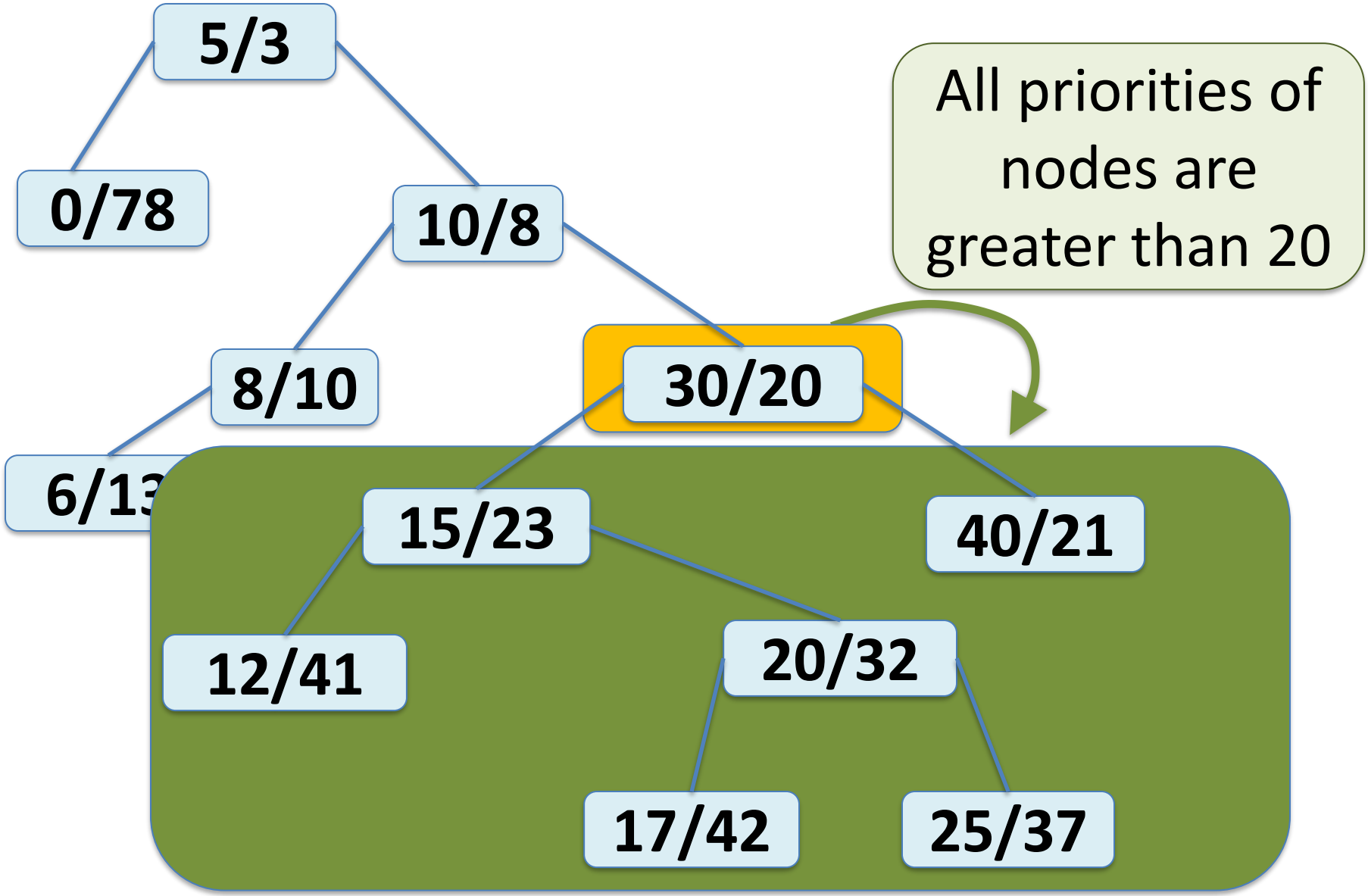
TREAP


5/3
All priorities of nodes are greater than 3
0/78
10/8
8/10
30/20
6/13
15/23
40/21
12/41
20/32
17/42
25/37

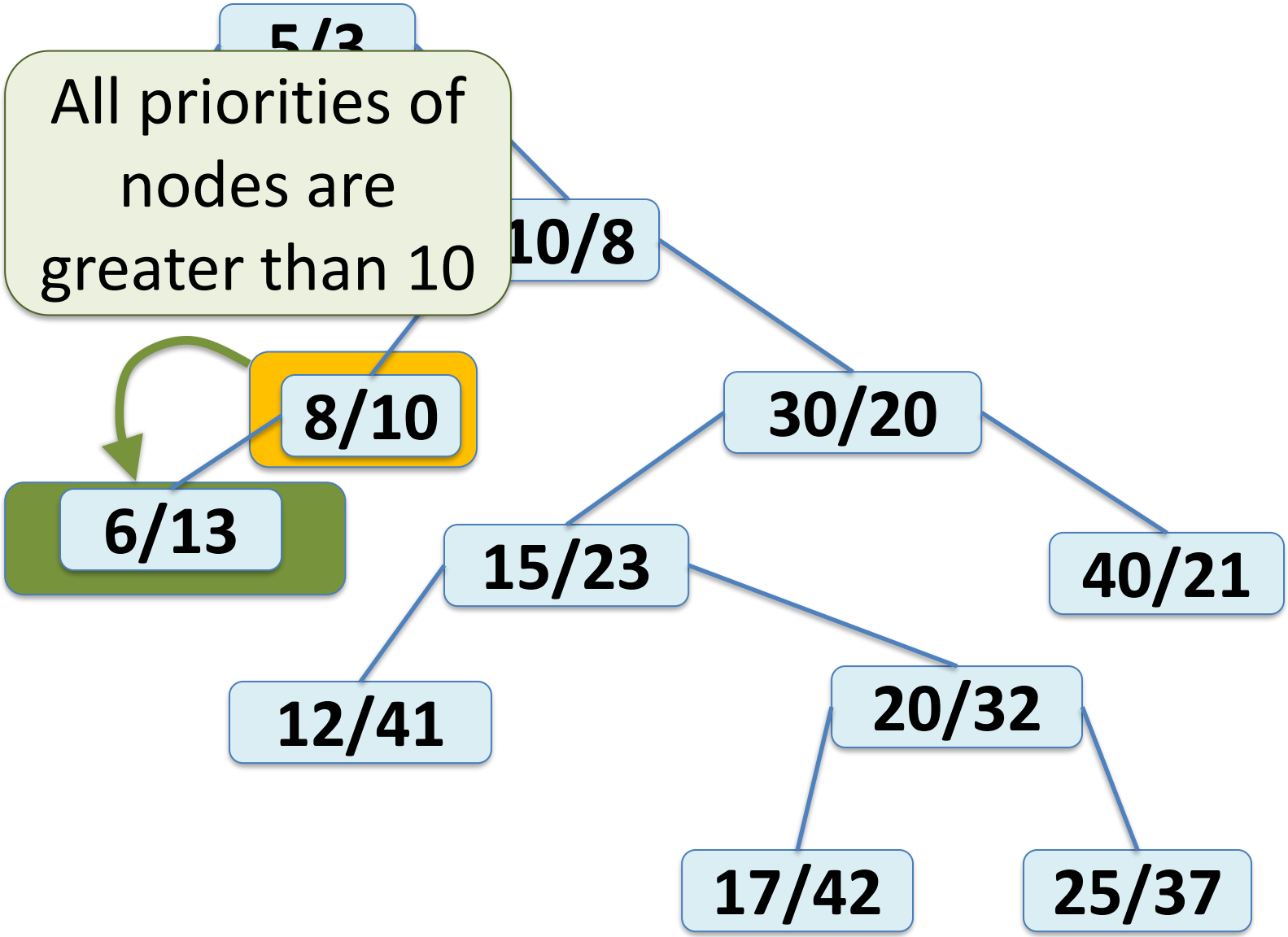
TREAP


5/3
0/78
10/8
All priorities of nodes are greater than 8
8/10
30/20
6/13
15/23
40/21
12/41
20/32
17/42
25/37

TREAP


5/3
0/78
10/8
8/10
30/20
6/13
15/23
40/21
12/41
20/32
17/42
25/37
All priorities of nodes are greater than 20

TREAP


All priorities of nodes are greater than 10
10/8
8/10
6/13
30/20
15/23
40/21
12/41
20/32
17/42
25/37

TREAP

How do we add a node to a treap?

TREAP

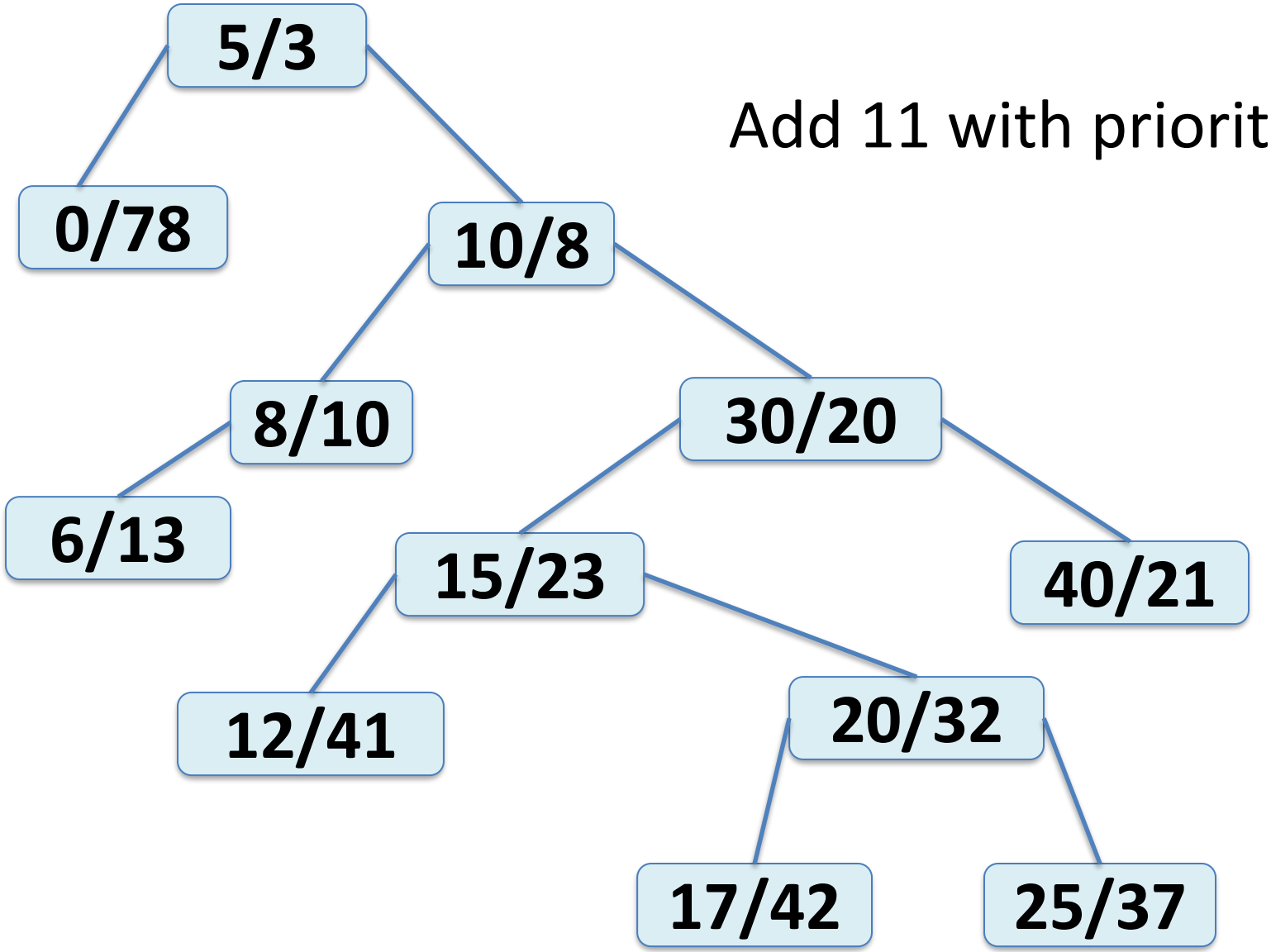
How do we add a node to a treap?

Observation:

Adding a new node, with a random (but unique) priority, in the way we have seen for binary trees will

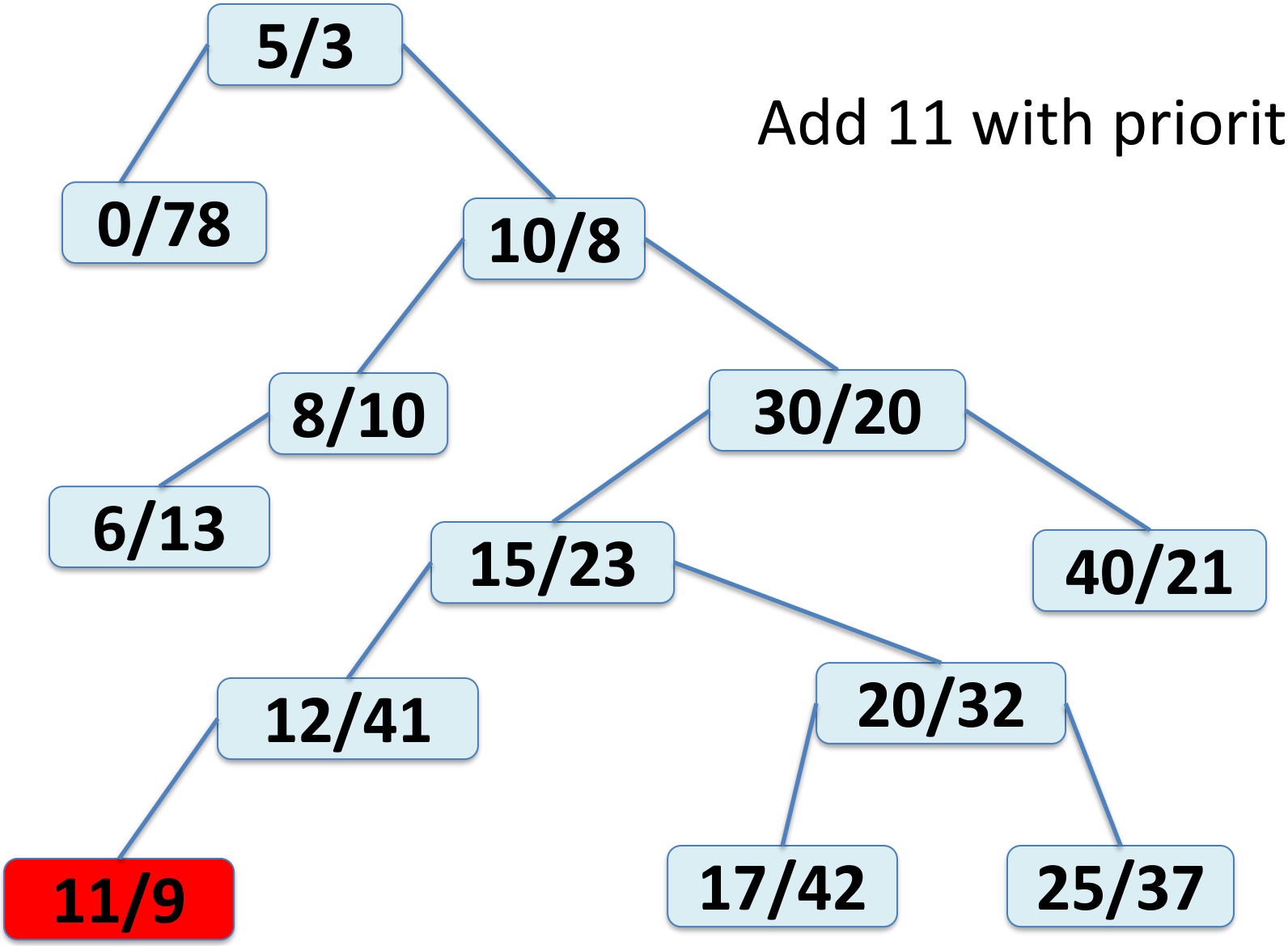
1. always maintain the binary search tree property,
2. likely not maintain the heap property.

TREAP



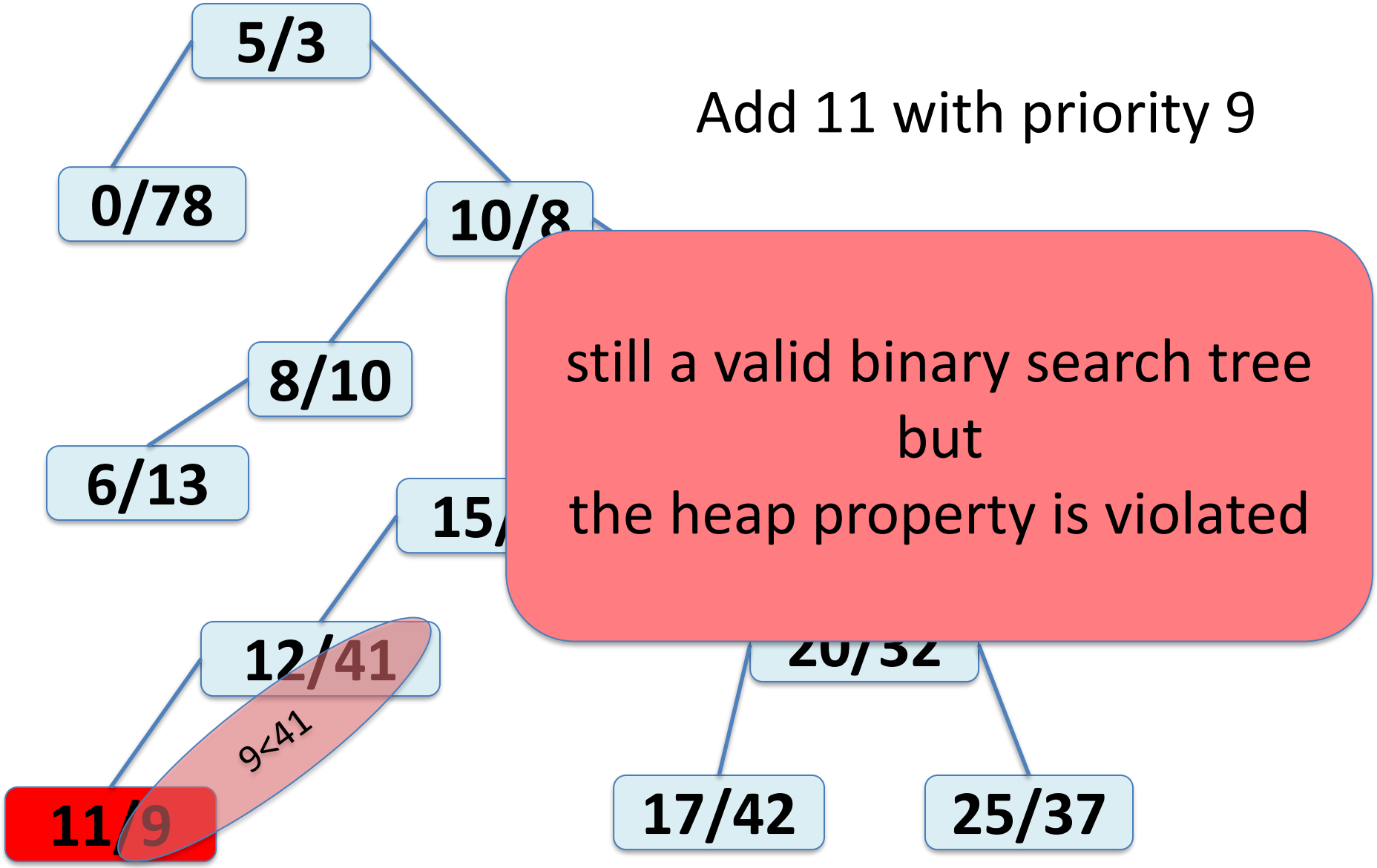
Add 11 with priority 9

TREAP



Add 11 with priority 9

TREAP


5/3
0/78
10/8
8/10
6/13
15/
12/41
9<41
11/9
20/32
17/42
25/37
Add 11 with priority 9
still a valid binary search tree
but
the heap property is violated

TREAP

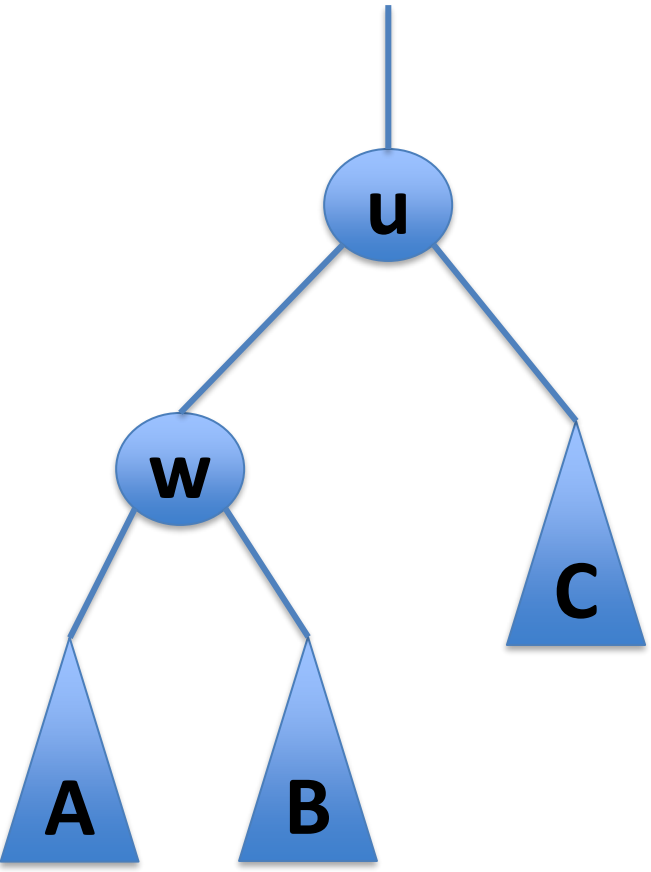
How can we modify a binary search tree while maintaining the binary search tree property?

There are many ways to do this. For example,

1. swap an element with its immediate successor
2. swap an element with its immediate predecessor

TREAP

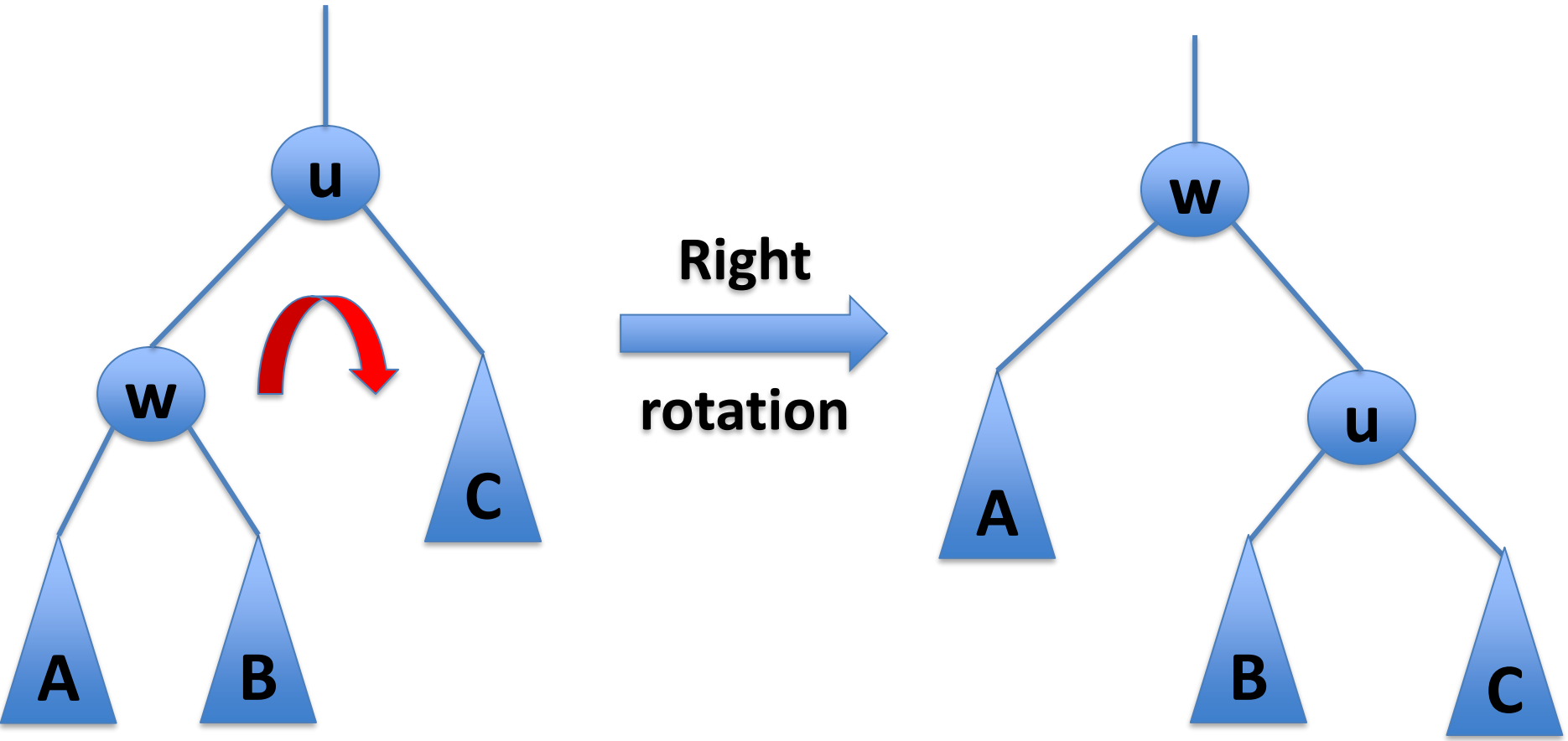
Let's consider a subtree rotation.



$$w.x < u.x$$
$$\forall a \in A : a.x < w.x$$
$$\forall b \in B : w.x < b.x < u.x$$
$$\forall c \in C : u.x < c.x$$

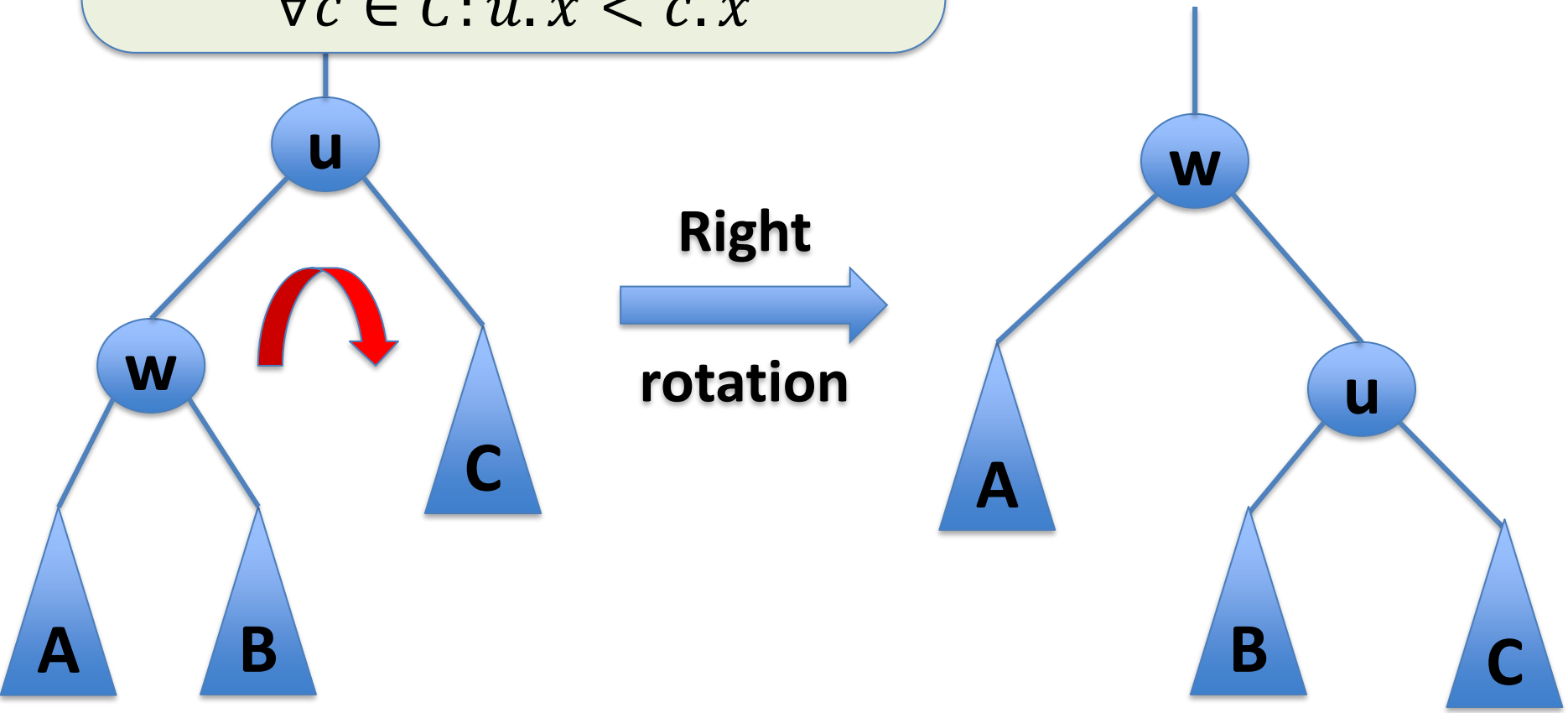
TREAP

Let's consider a subtree rotation.

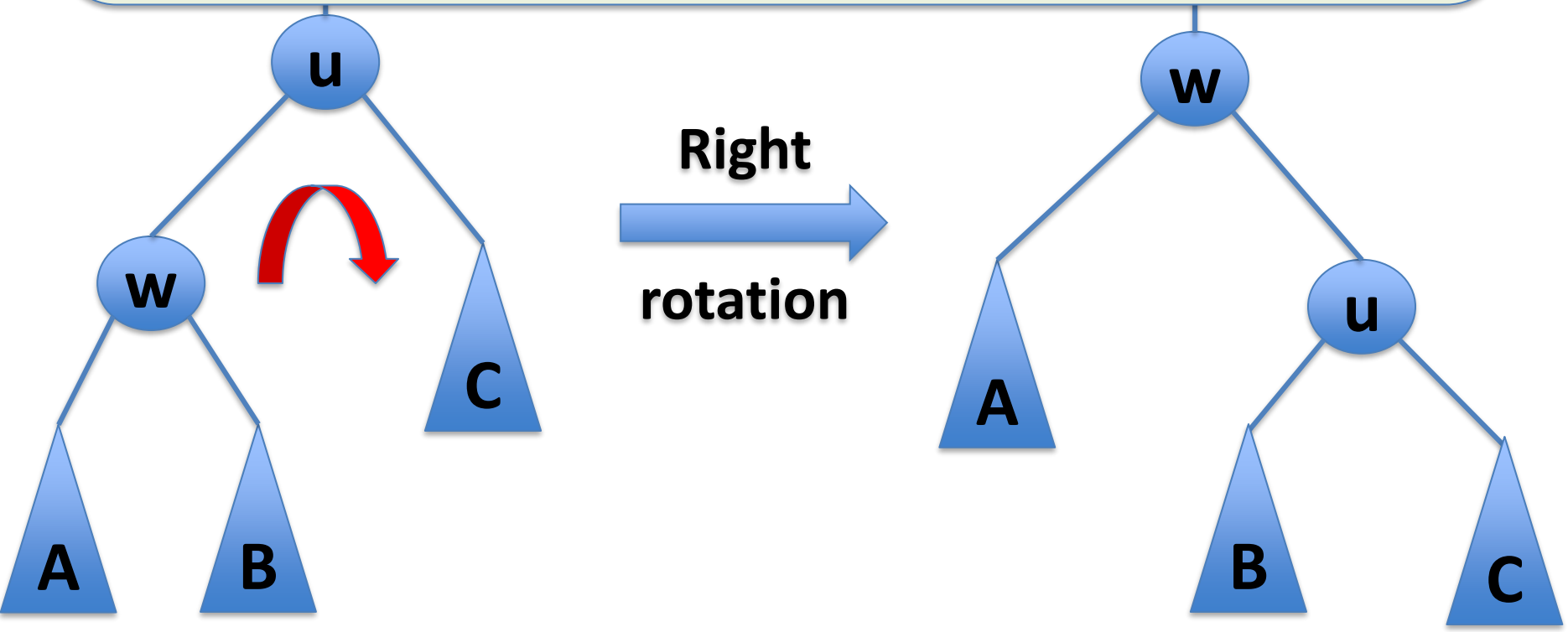


L n.

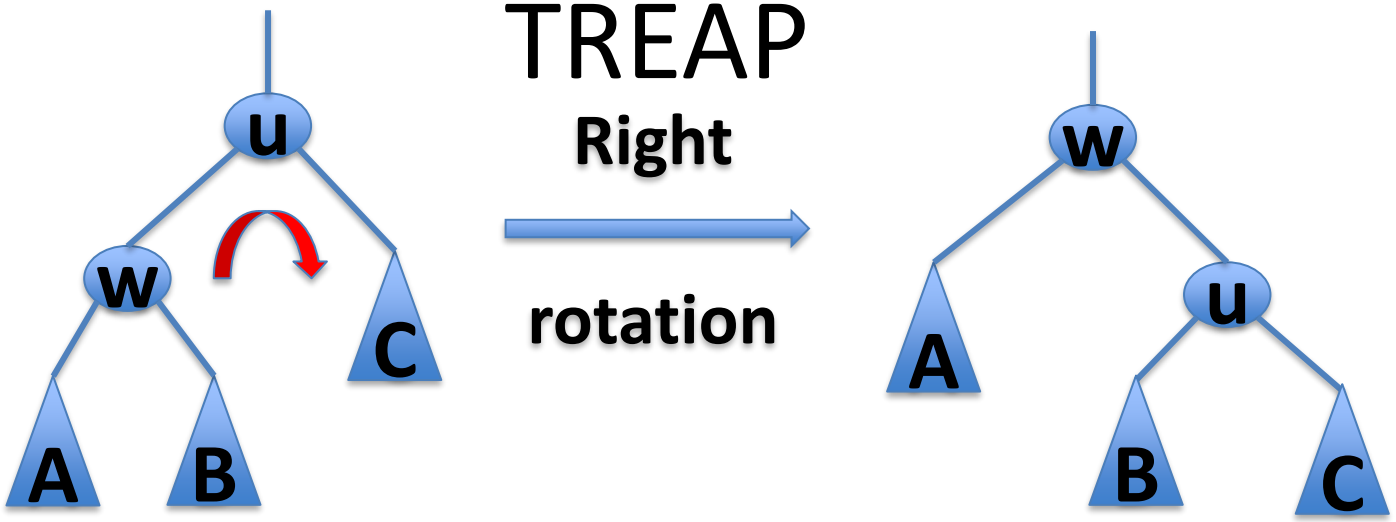
$$w.x < u.x$$
$$\forall a \in A : a.x < w.x$$
$$\forall b \in B : w.x < b.x < u.x$$
$$\forall c \in C : u.x < c.x$$



After the rotation, A is still below w, C is still below u, and B is still below both u and w. But, u and w are swapped. (this can help fix the heap order!)



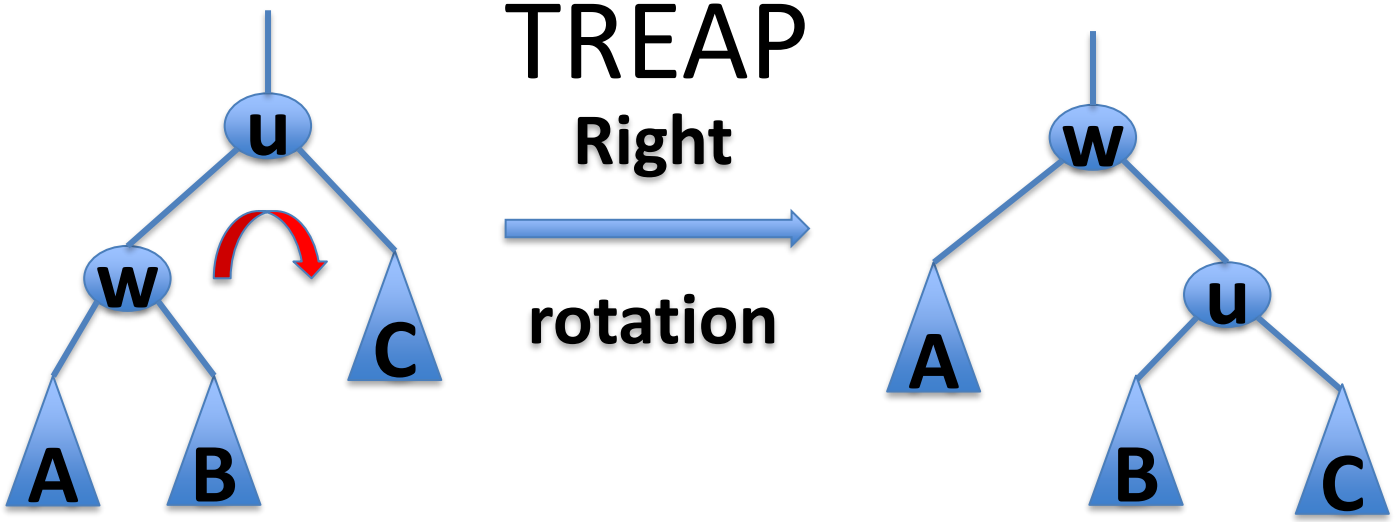
TREAP



What can we say about the dept of all the nodes after a rotation?

What can we say about the height of the tree?

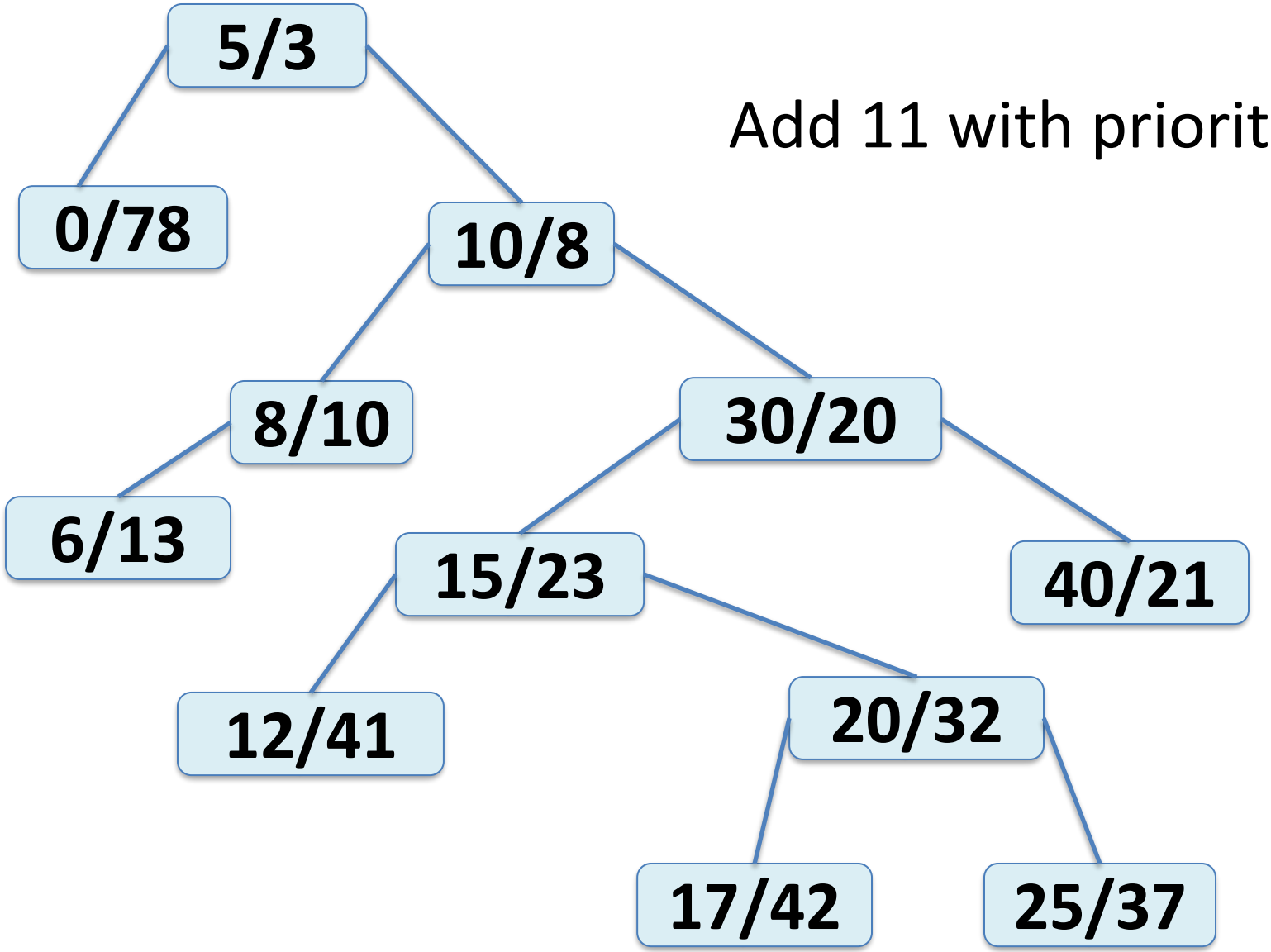
TREAP



$$depth(u) \rightarrow +1$$
$$depth(w) \rightarrow -1$$
$$depth(a \in A) \rightarrow -1$$
$$depth(b \in B) \rightarrow unchanged$$
$$depth(c \in C) \rightarrow +1$$

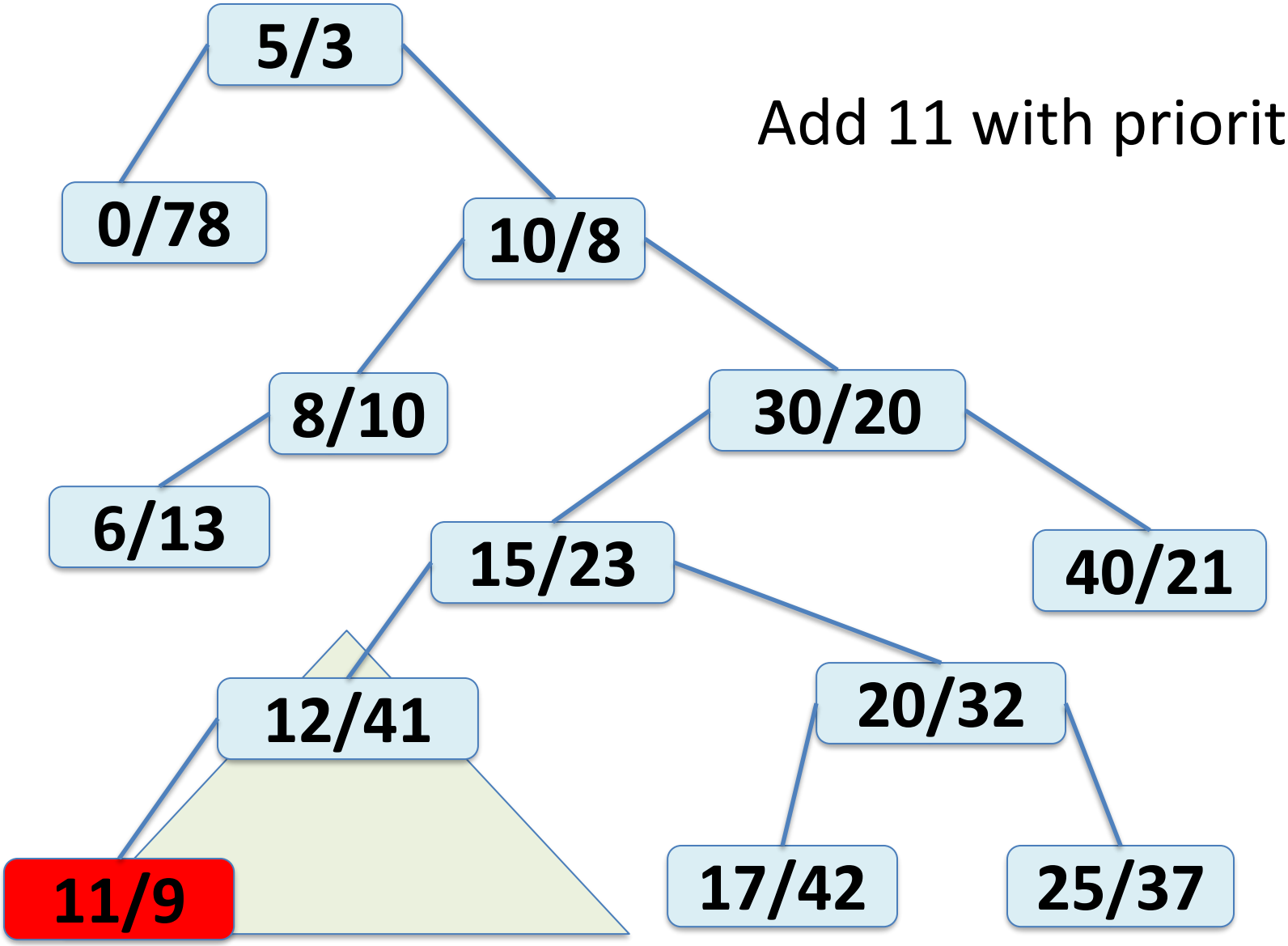
$$height(tree) \rightarrow \{-1,0,+1\}$$

TREAP



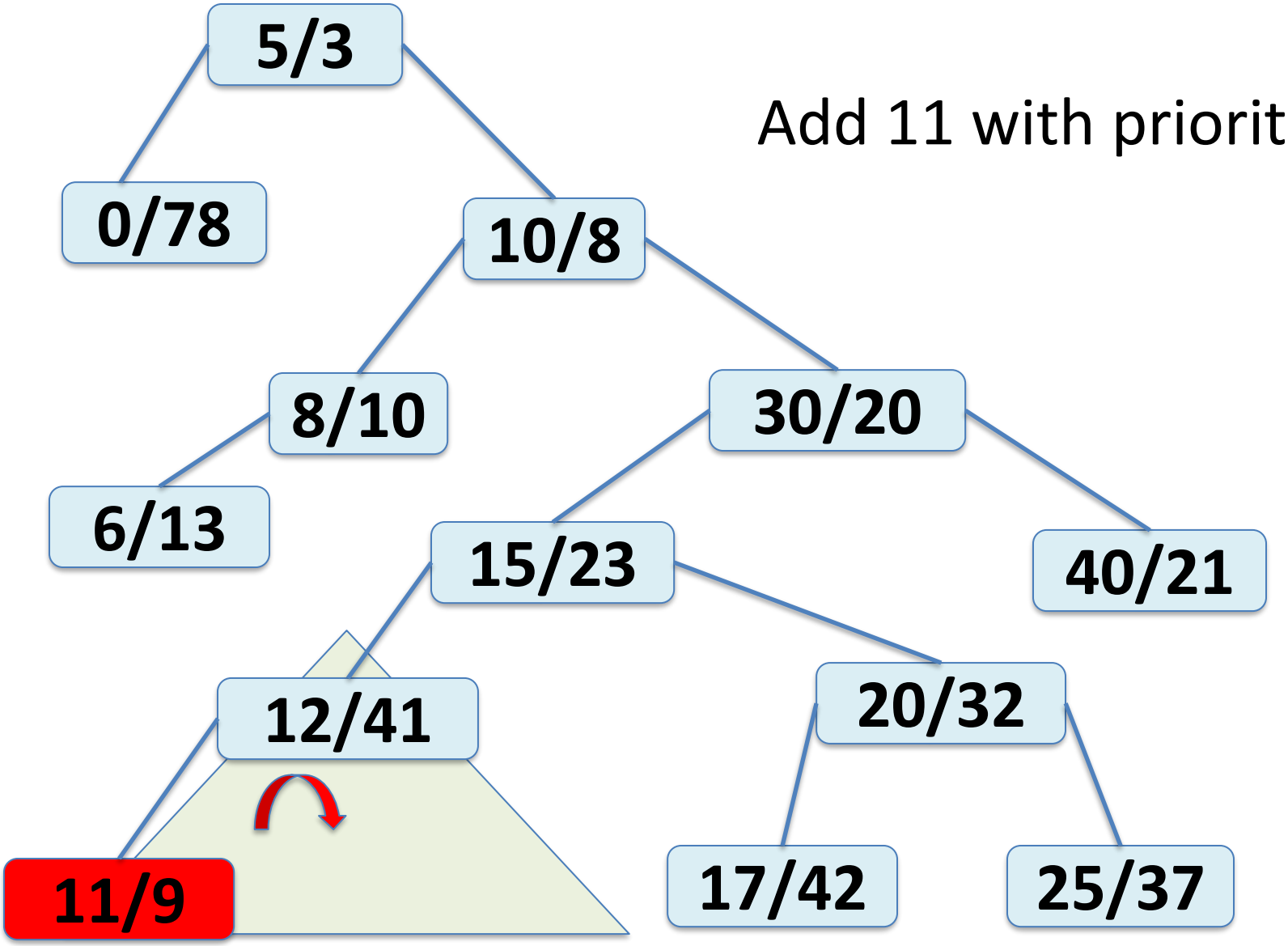
Add 11 with priority 9

TREAP



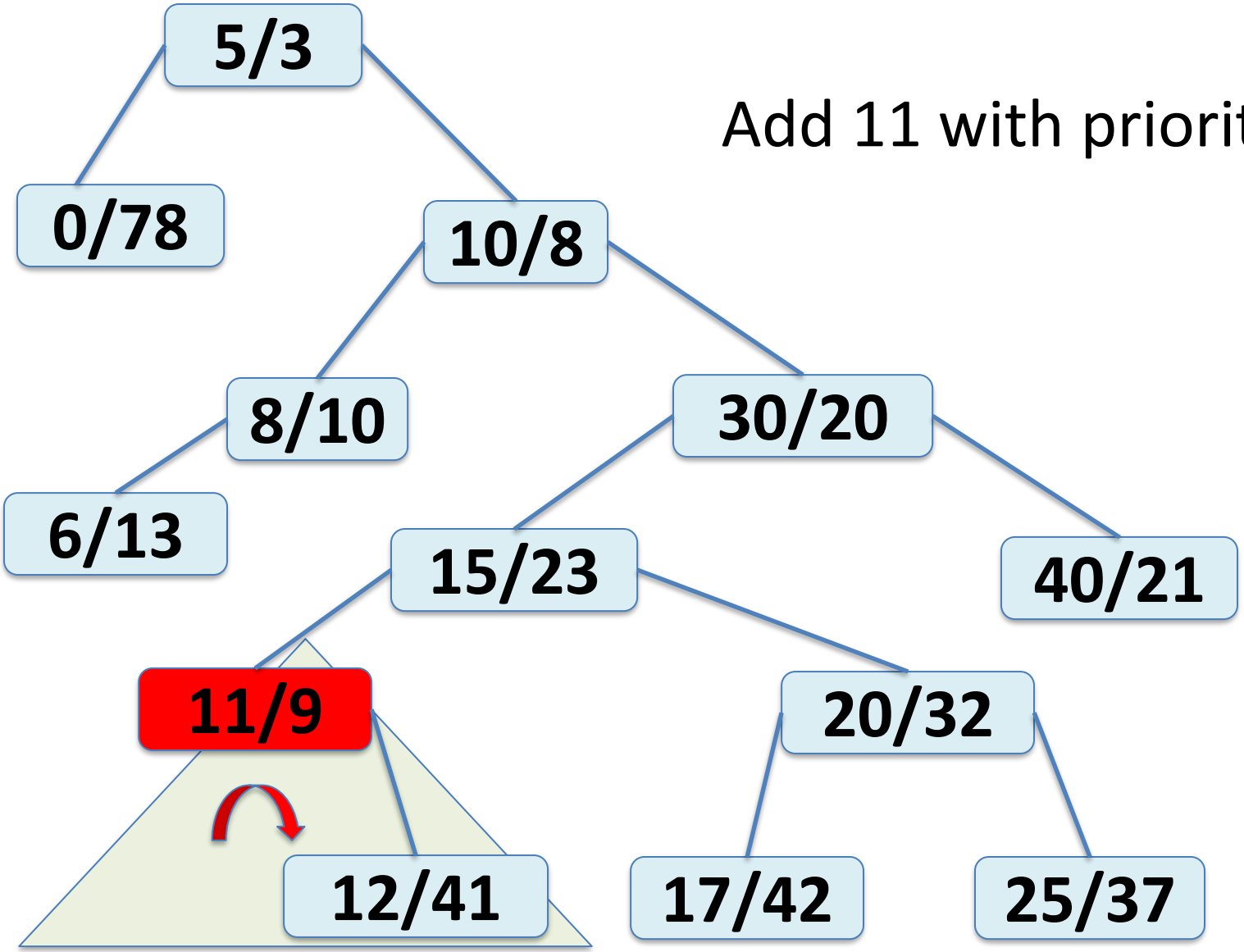
Add 11 with priority 9

TREAP



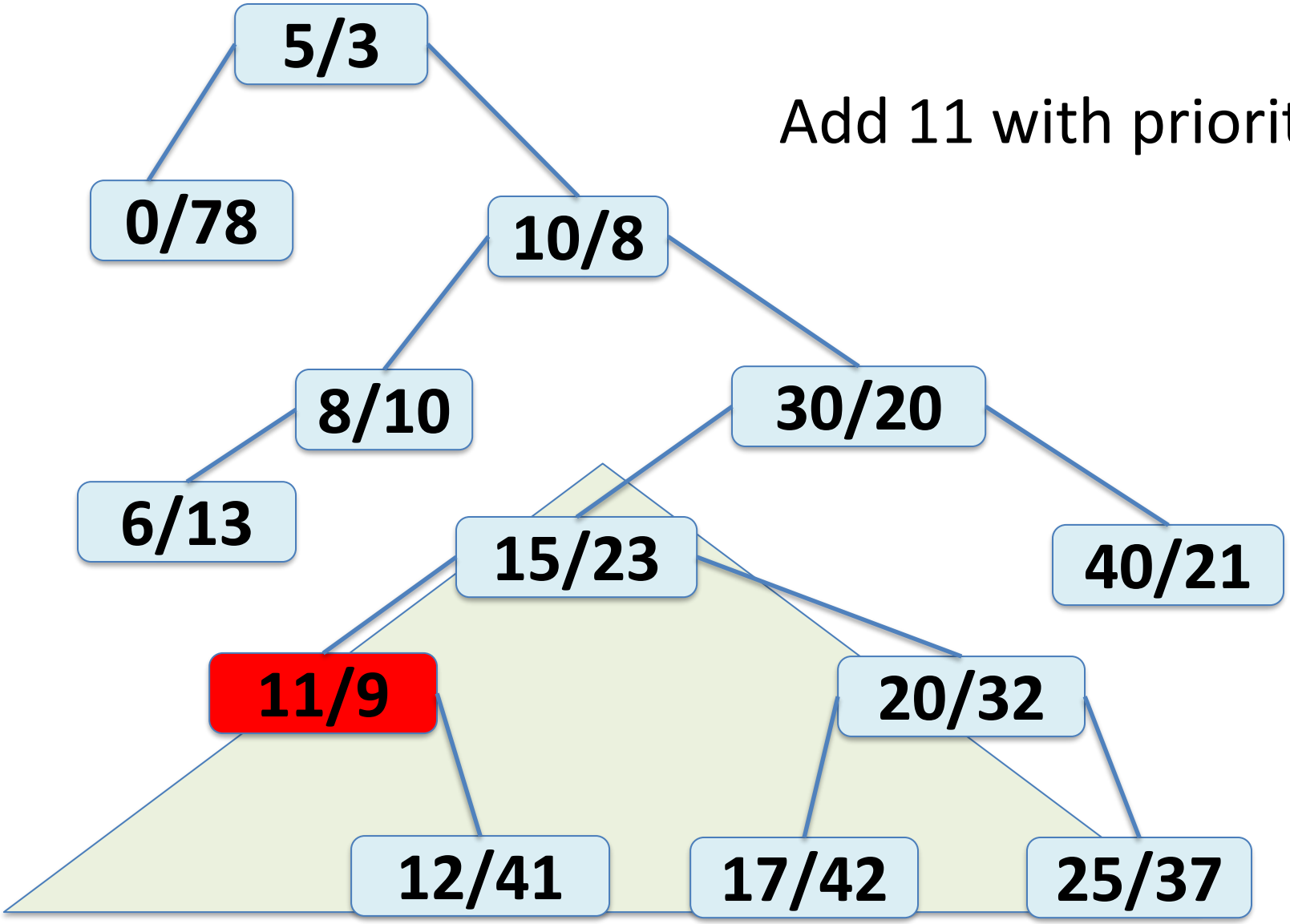
Add 11 with priority 9

TREAP



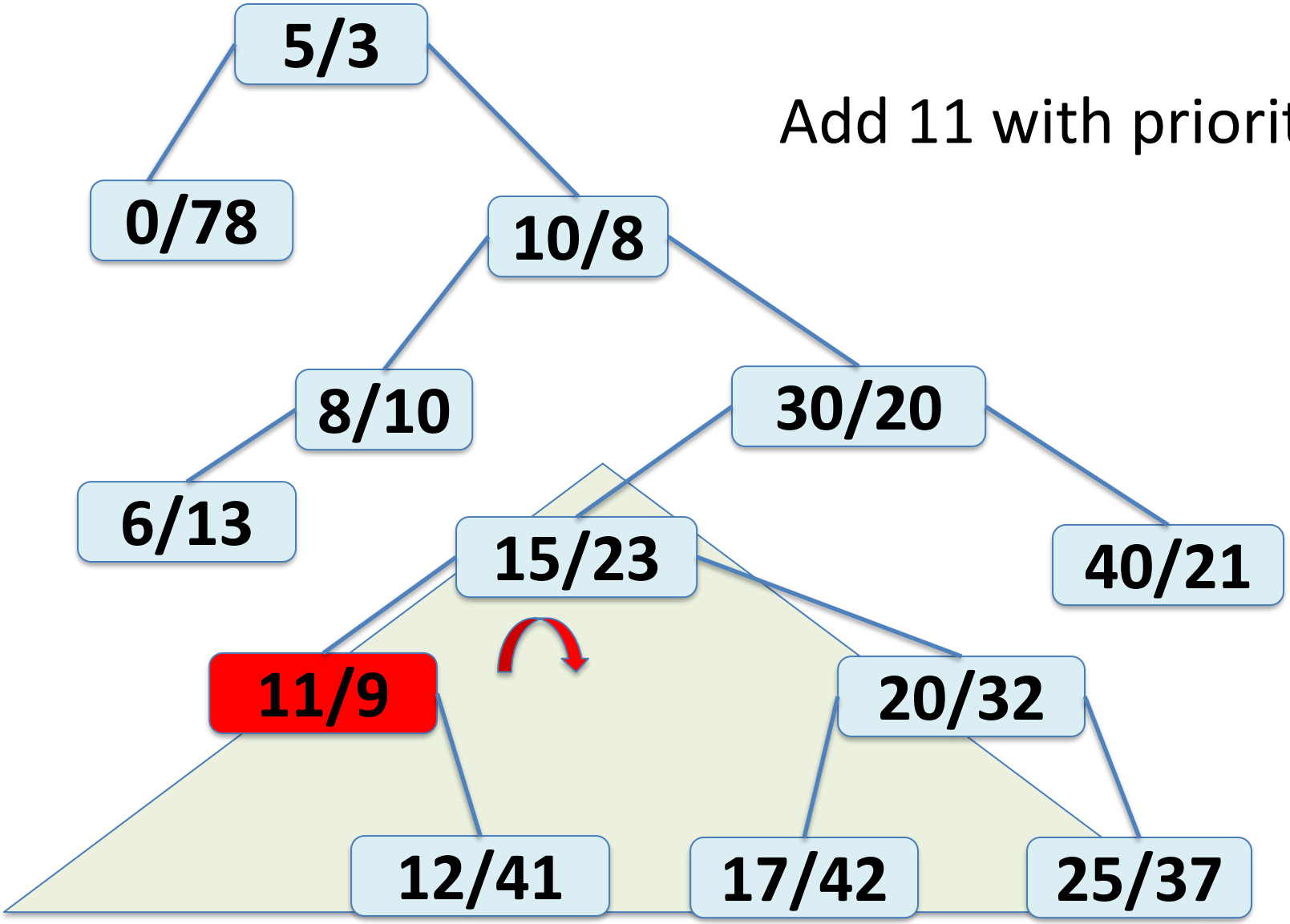
Add 11 with priority 9

TREAP



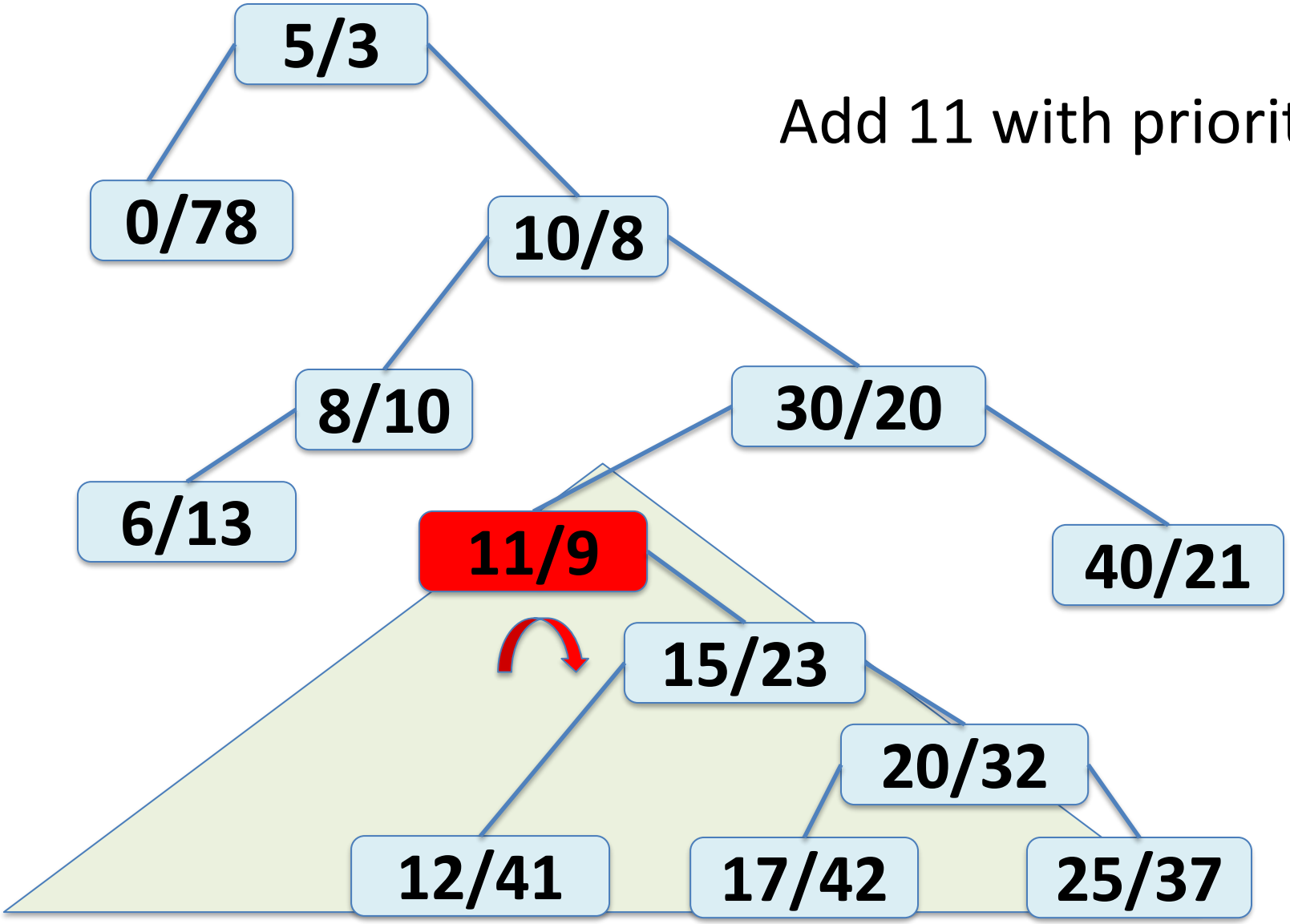
Add 11 with priority 9

TREAP



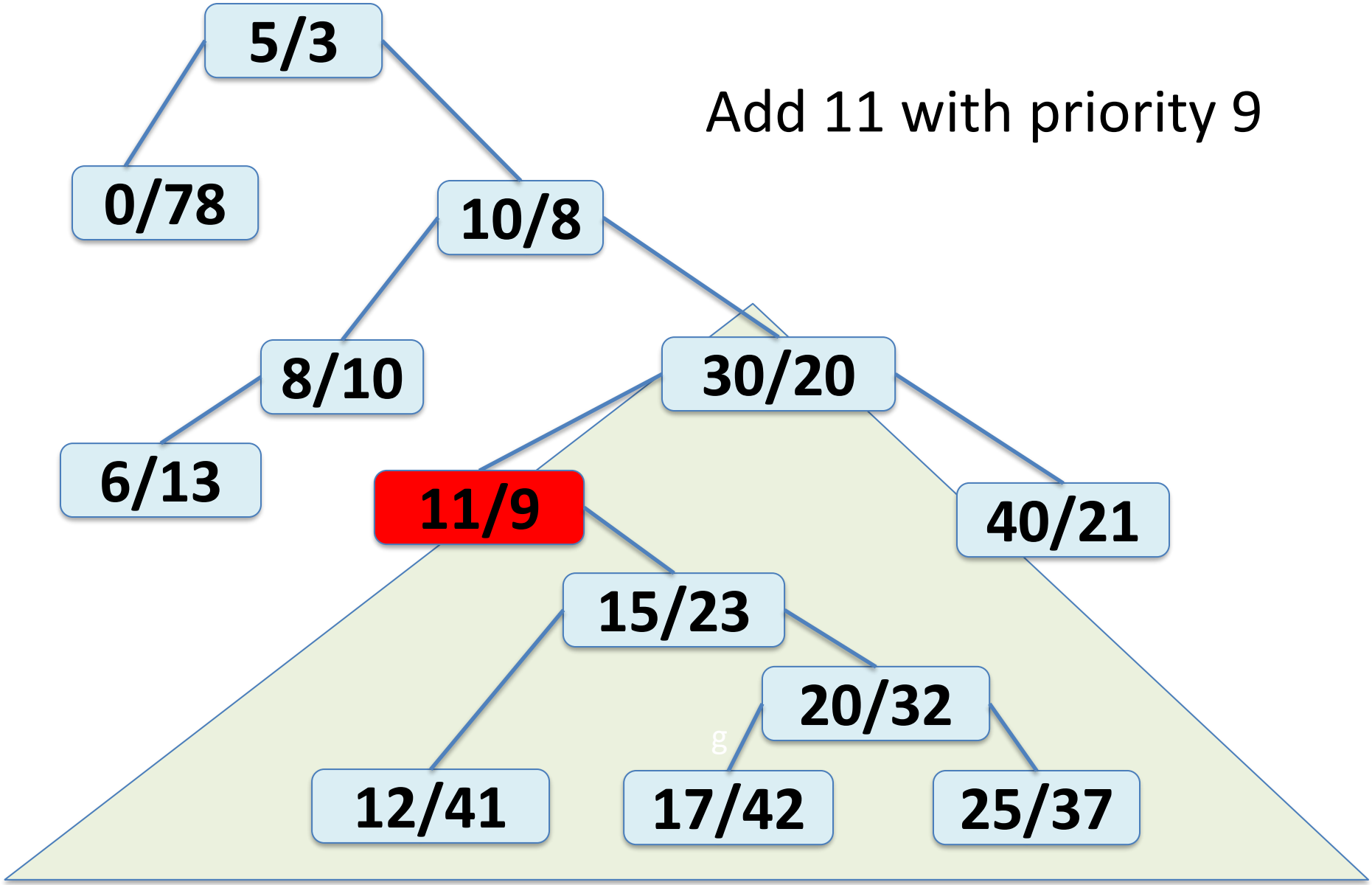
Add 11 with priority 9

TREAP

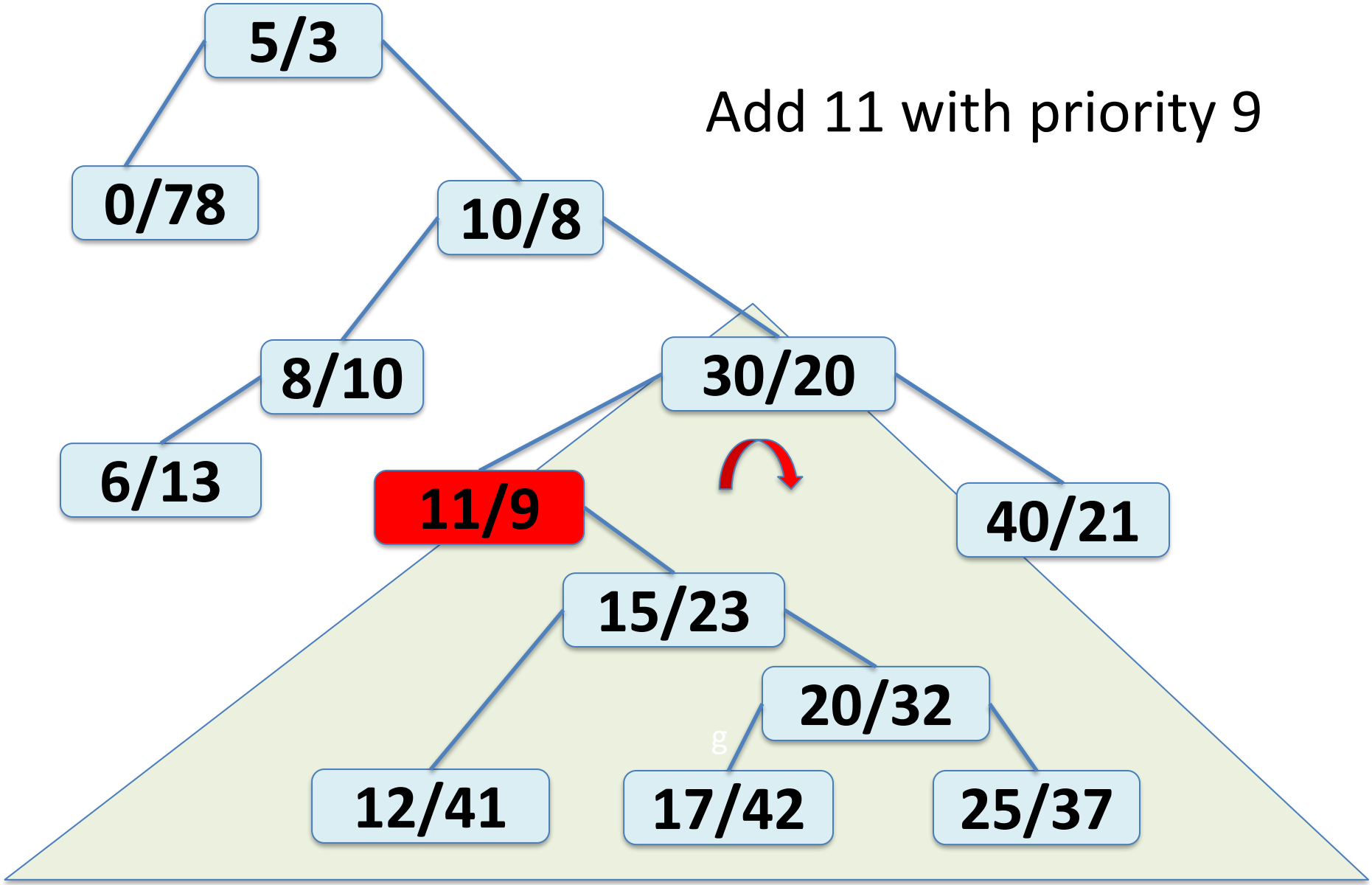


Add 11 with priority 9

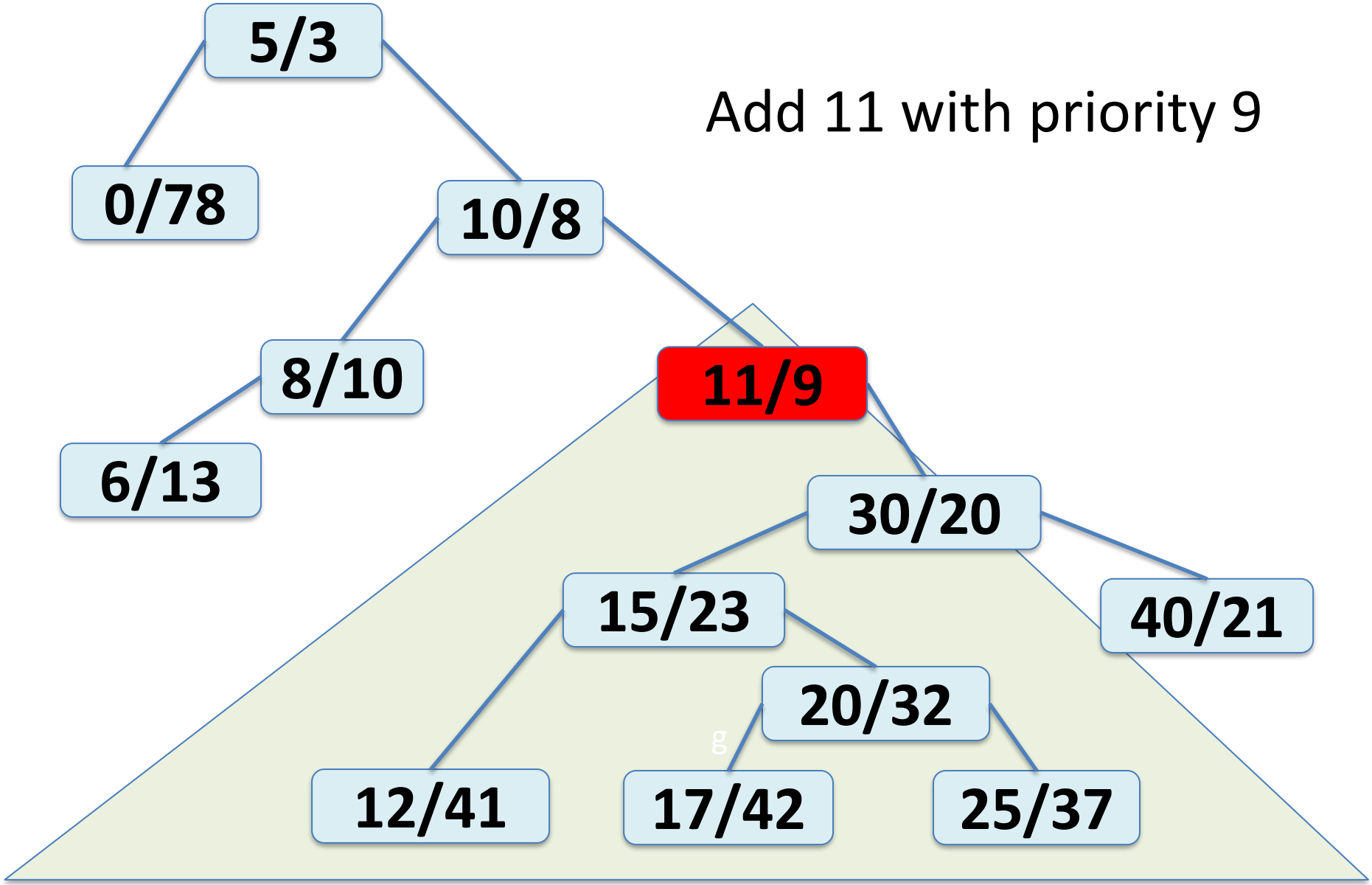
TREAP


5/3
0/78
10/8
8/10
30/20
6/13
11/9
40/21
15/23
20/32
12/41
17/42
25/37
Add 11 with priority 9

TREAP

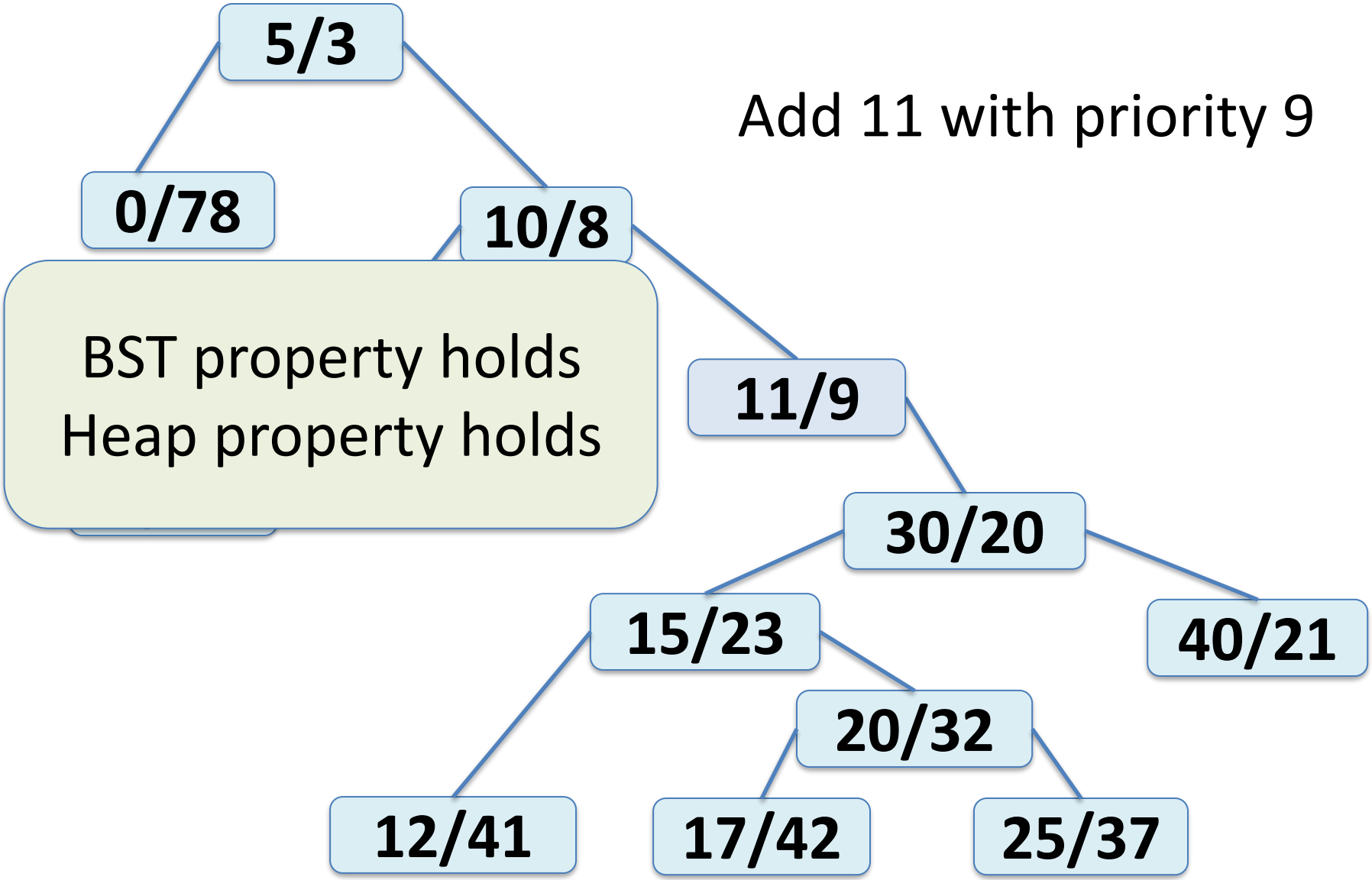

5/3
0/78
10/8
8/10
30/20
6/13
11/9
40/21
15/23
20/32
12/41
17/42
25/37
Add 11 with priority 9

TREAP

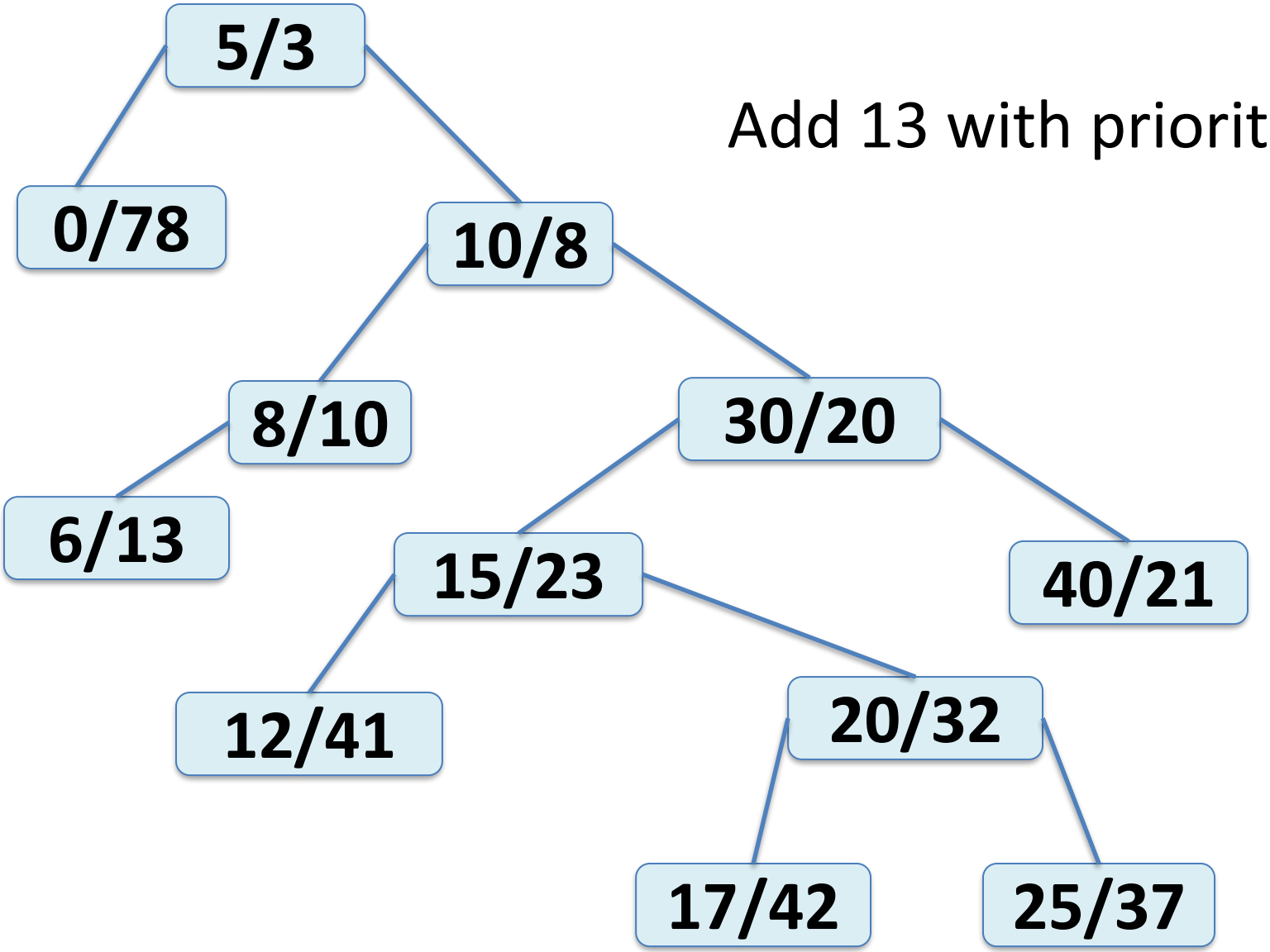


Add 11 with priority 9

TREAP

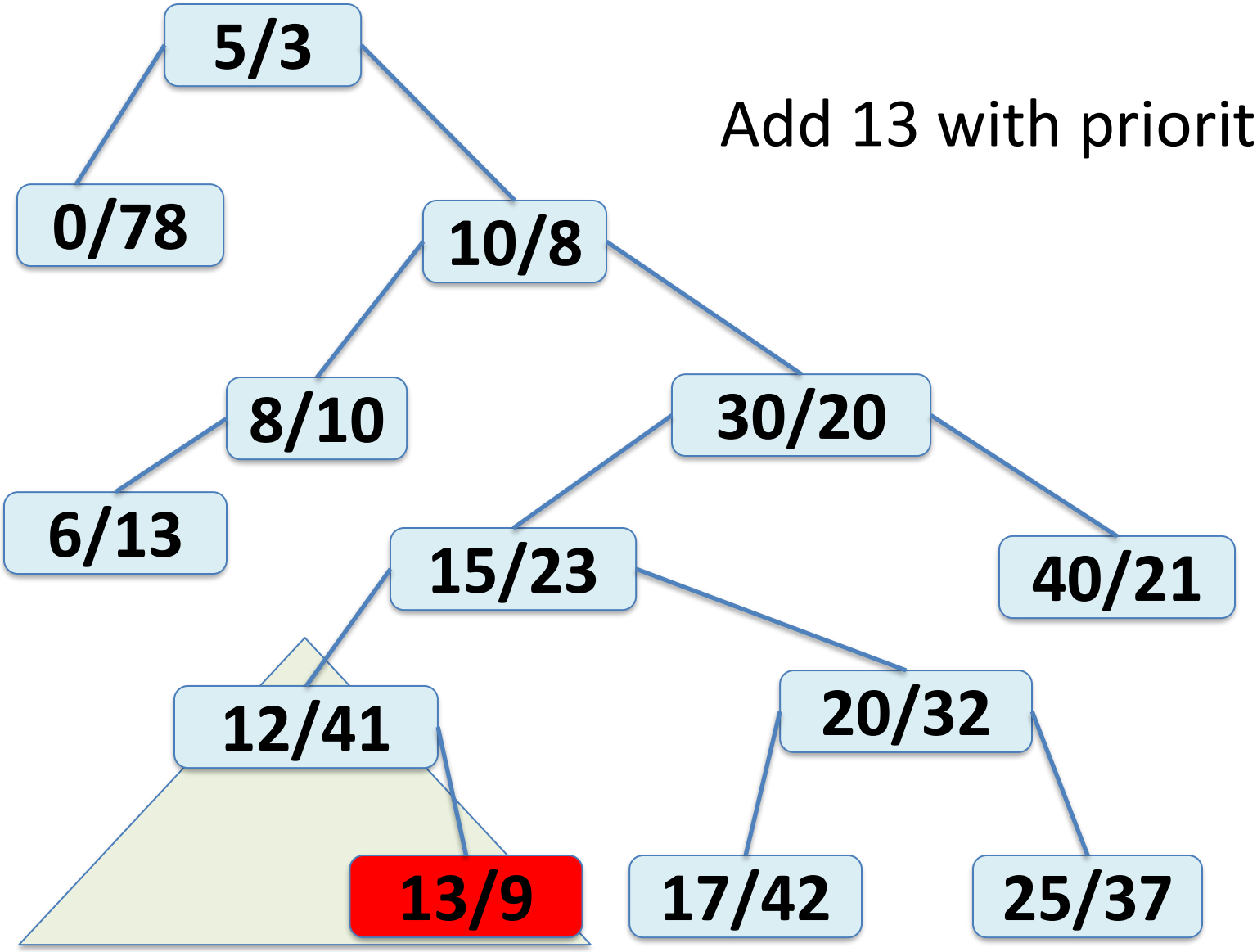

5/3
0/78
10/8
11/9
30/20
15/23
40/21
20/32
12/41
17/42
25/37
Add 11 with priority 9
BST property holds
Heap property holds

TREAP



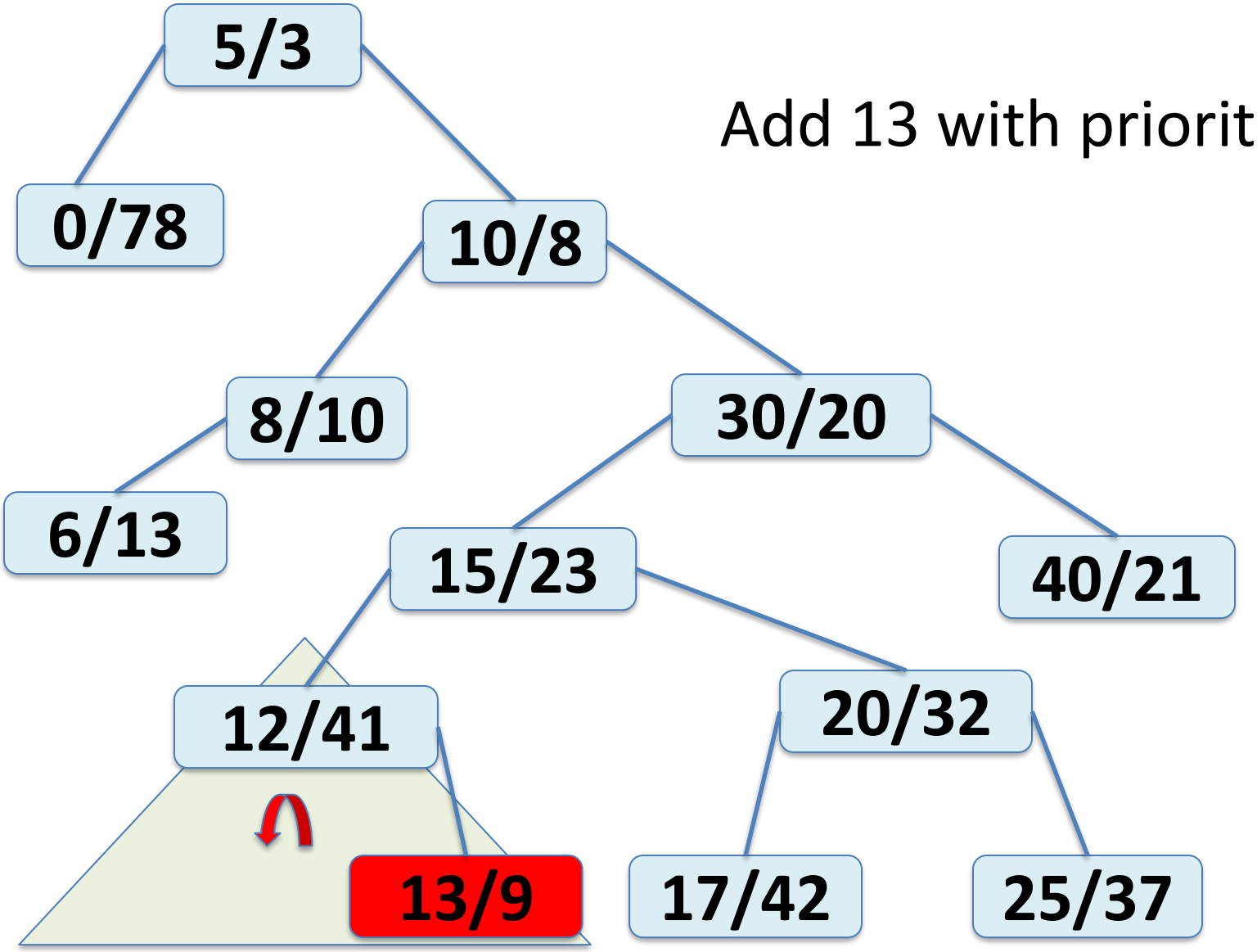
Add 13 with priority 9

TREAP



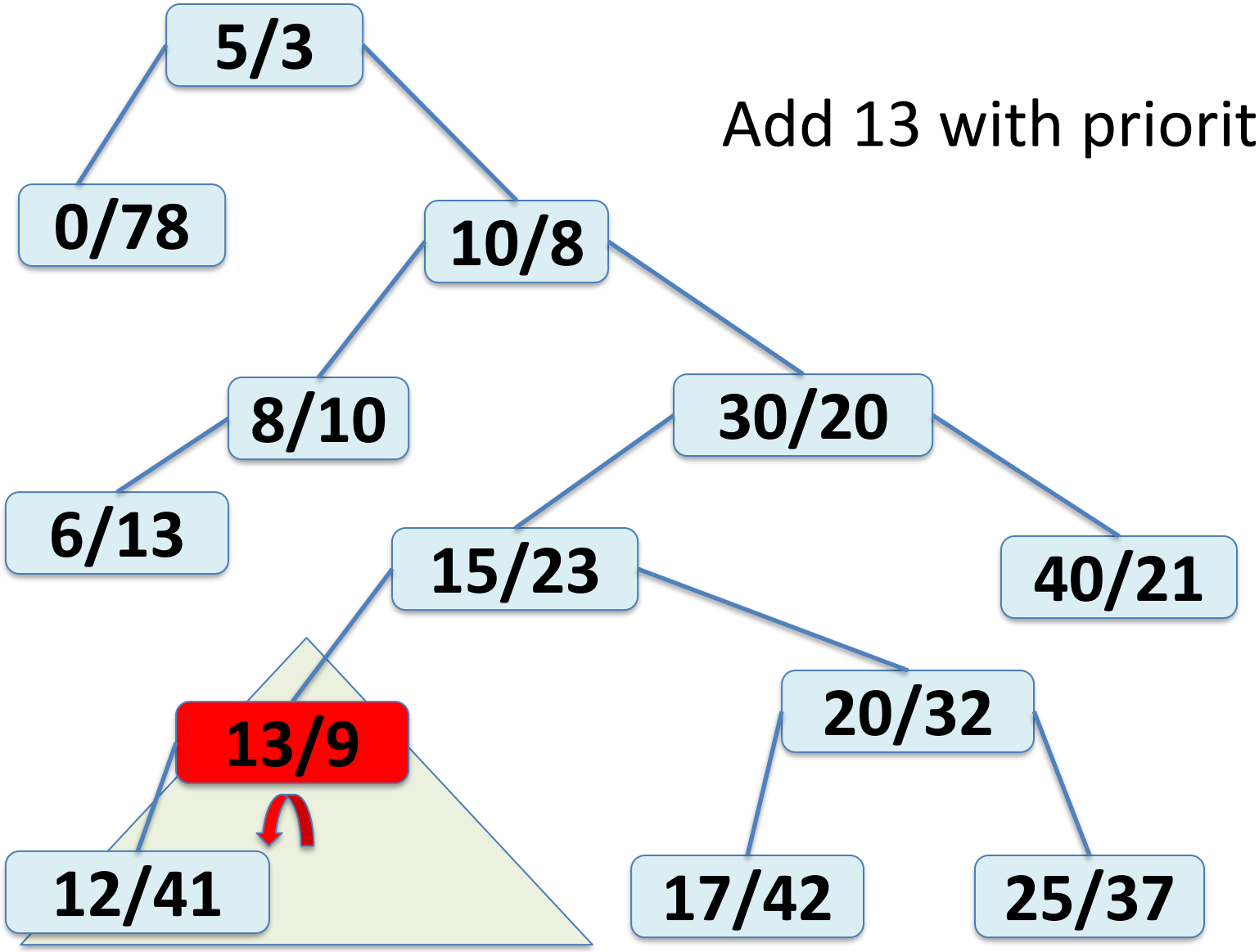
Add 13 with priority 9

TREAP



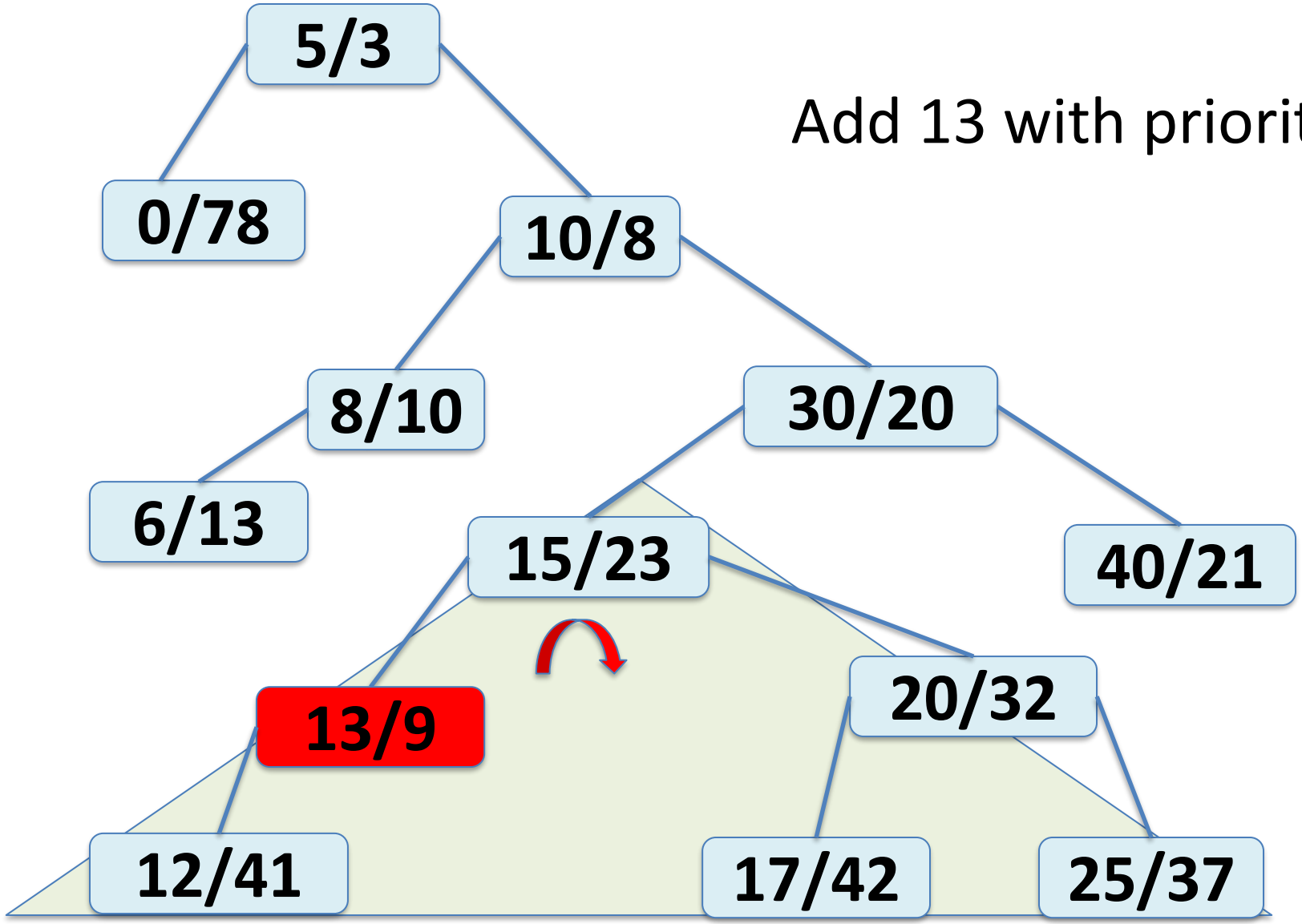
Add 13 with priority 9

TREAP



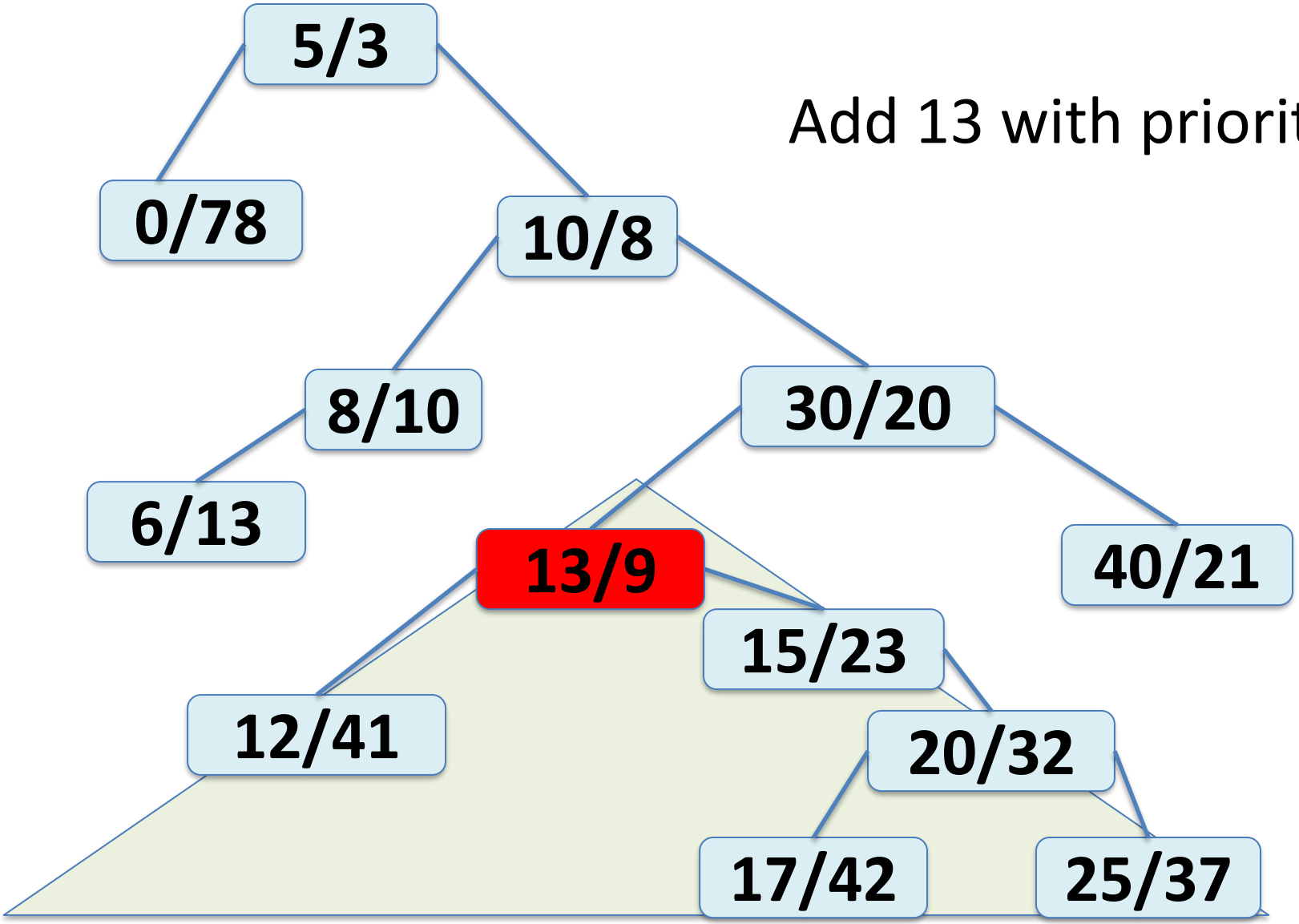
Add 13 with priority 9

TREAP



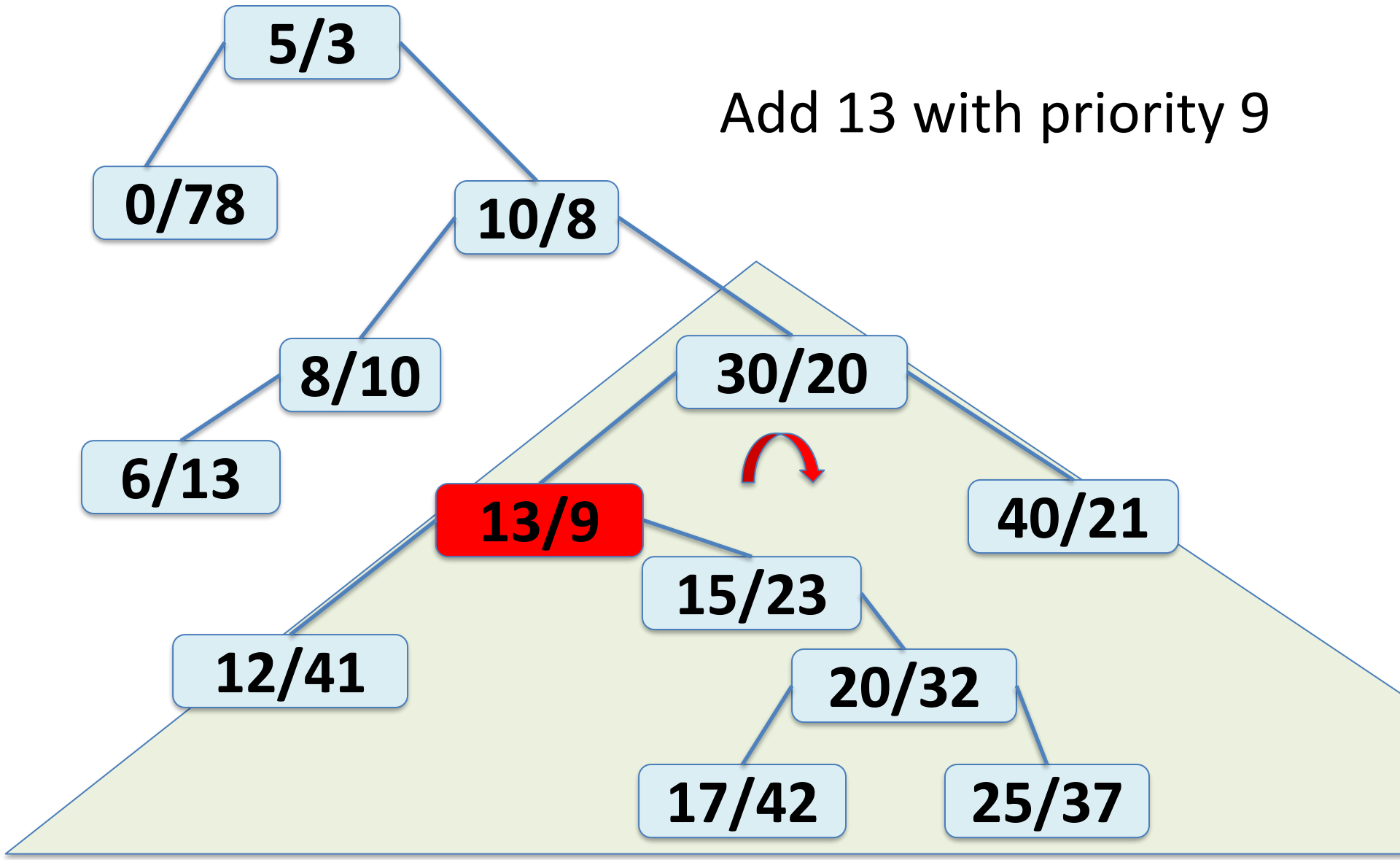
Add 13 with priority 9

TREAP

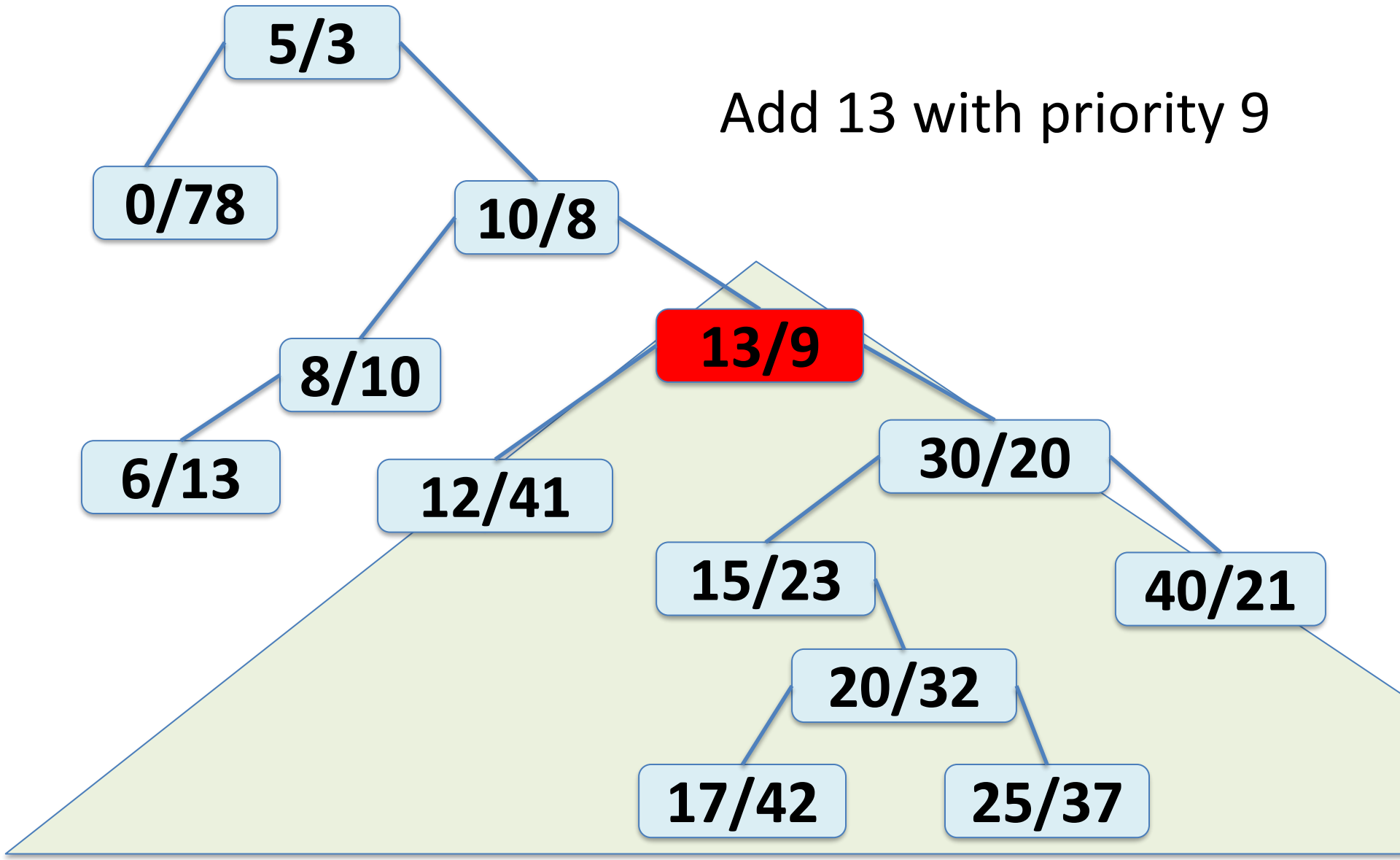


Add 13 with priority 9

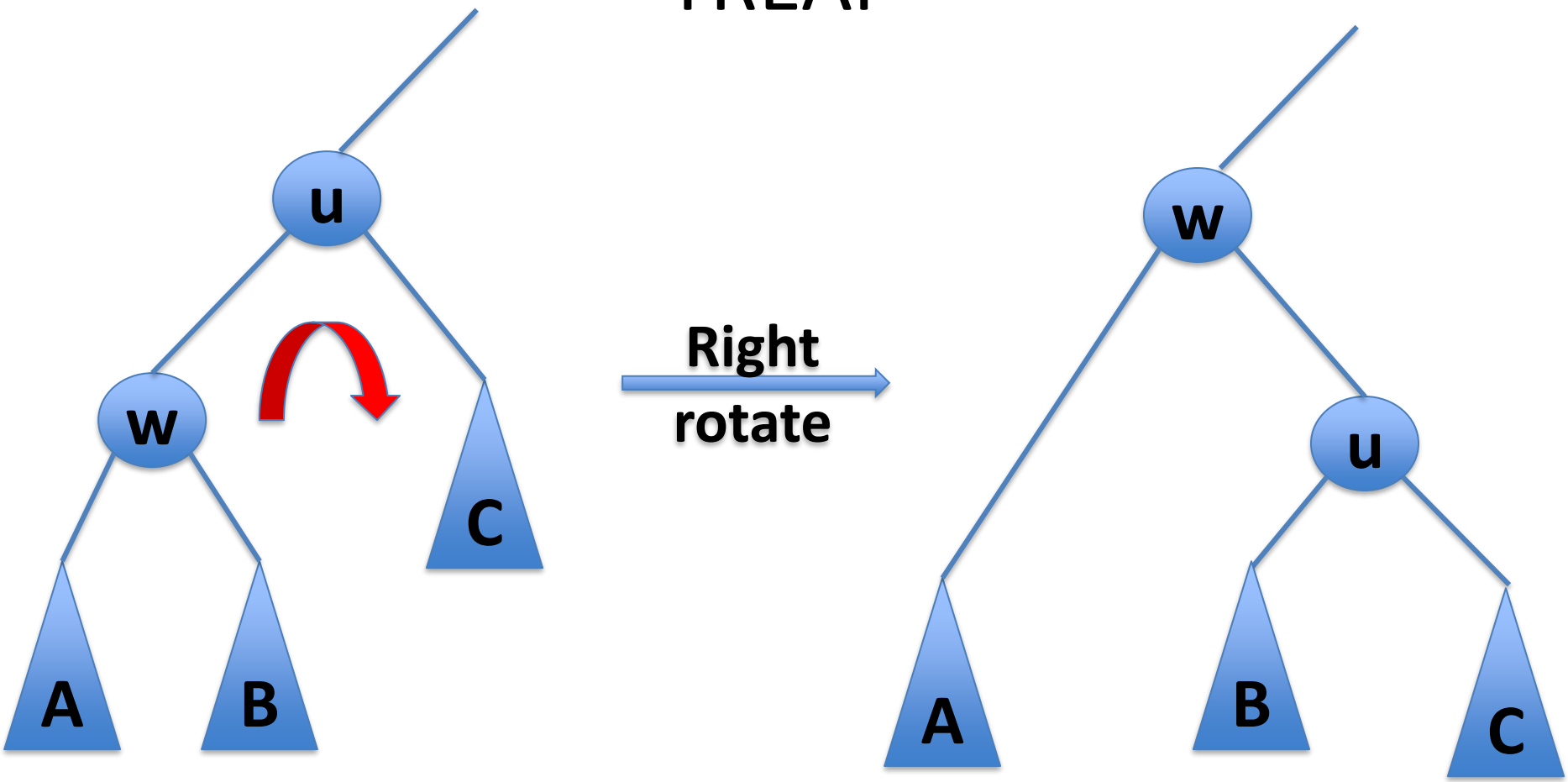
TREAP


5/3
0/78
10/8
8/10
30/20
6/13
13/9
40/21
15/23
12/41
20/32
17/42
25/37
Add 13 with priority 9

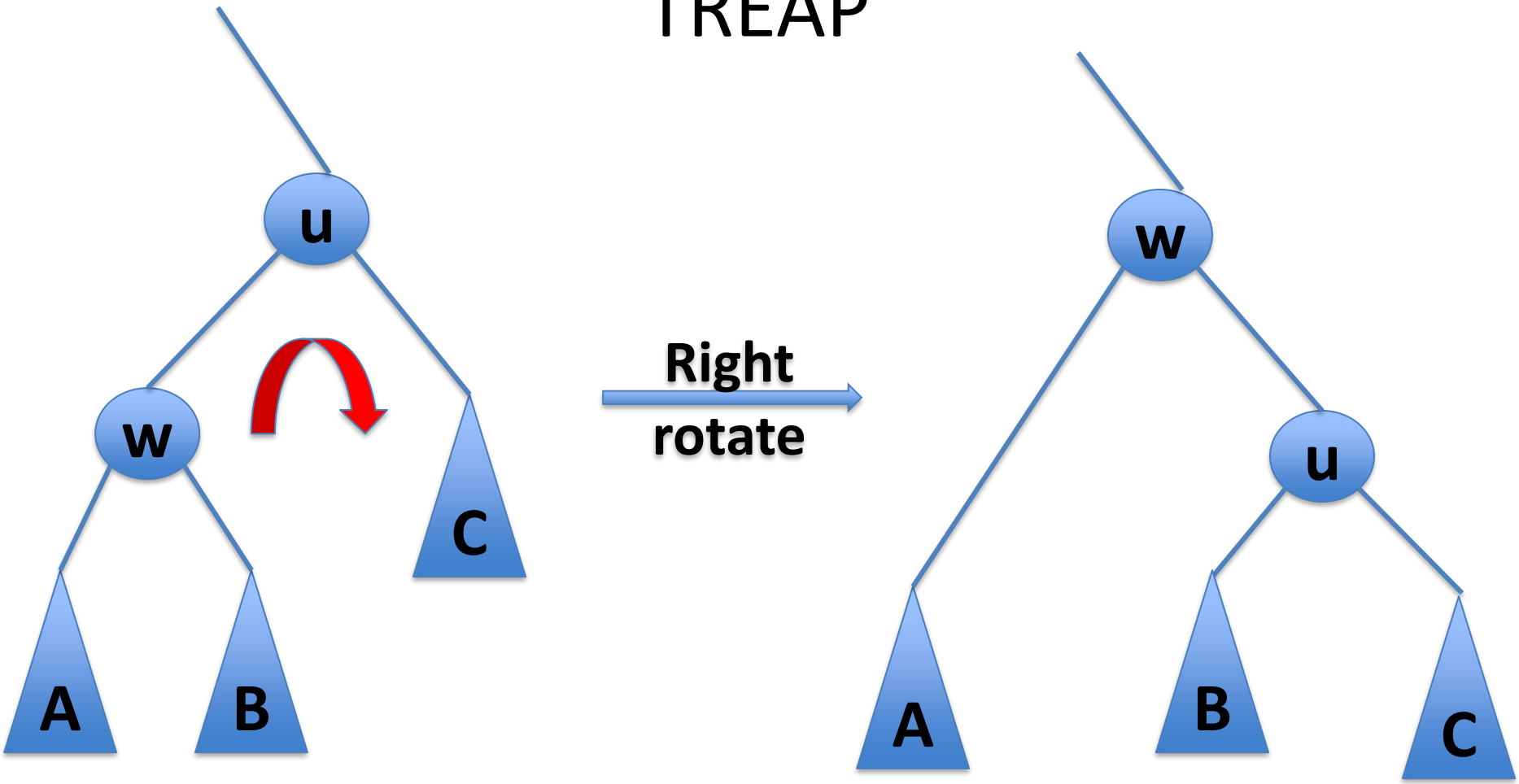
TREAP


5/3
0/78
10/8
8/10
13/9
6/13
12/41
30/20
15/23
40/21
20/32
17/42
25/37
Add 13 with priority 9

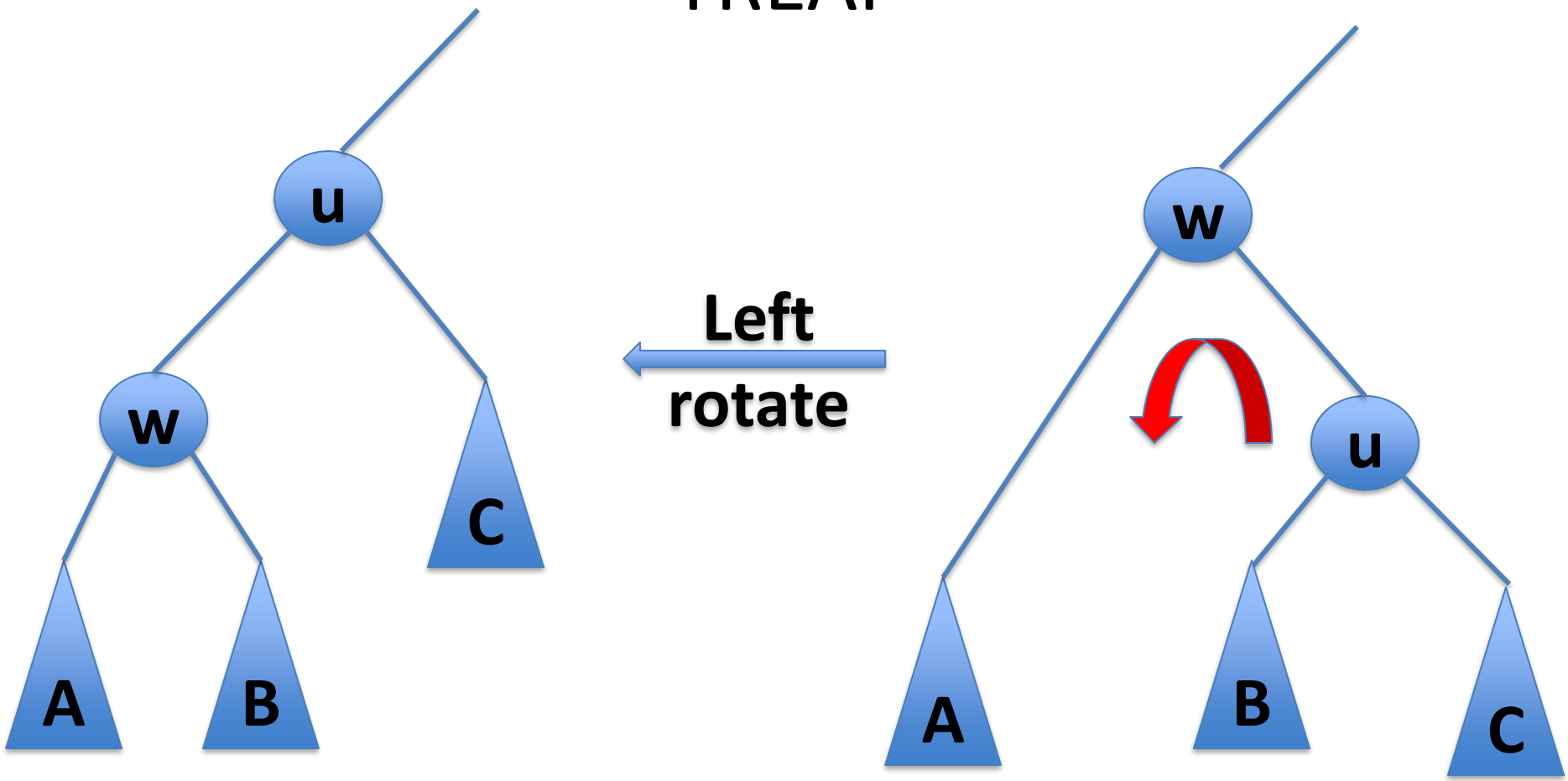
TREAP



TREAP



TREAP



TREAP

How do we add to a treap?

1. Insert as leaf (as usual for a binary search tree)

2. Rotate the new node **up** until the heap property of the treap is satisfied

What is the cost of adding to a treap?

TREAP

How do we remove from a treap?

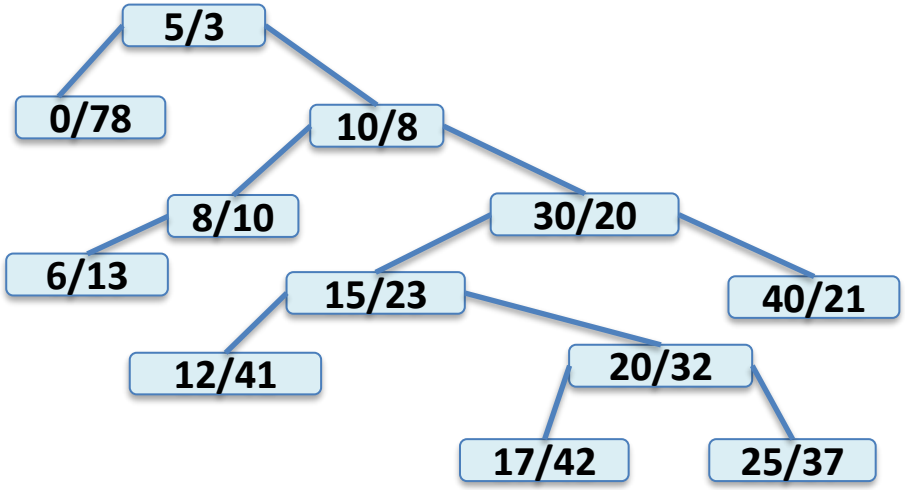
1. Find the element to delete.

2. Rotate the node **down** in the tree until it is a leaf and then splice it out of the tree.

What is the cost of removing from a treap?

TREAP

We can think of a treap as a BST in which the items were added in increasing order of priority.



< (5,3),(10,8),(8,10),(6,13),(30,20),(40,21),(15,23),
(20,32),(25,37),(12,41),(17,42),(0,78)>

< 5,10,8,6,30,40,15,20,25,12,17,0 >

TREAP

We can think of a treap as a BST in which the items were added in increasing order of priority.

Since the priorities are randomly assigned, this is the same as adding the elements based by value in a random order. But this is exactly what the random binary search tree did.

Therefore, the expected length of search path in a **treap** is the same as for a random binary search tree: at most

$$2\ln(n) + O(1) \approx 1.386\log(n) + O(1)$$

SKIPLIST vs TREAP

Expected length of search path in **skiplist** is at most

$$2\log(n) + O(1)$$

Expected length of search path in **treap** is at most

$$2\ln(n) + O(1) \approx 1.386\log(n) + O(1)$$

TREAP

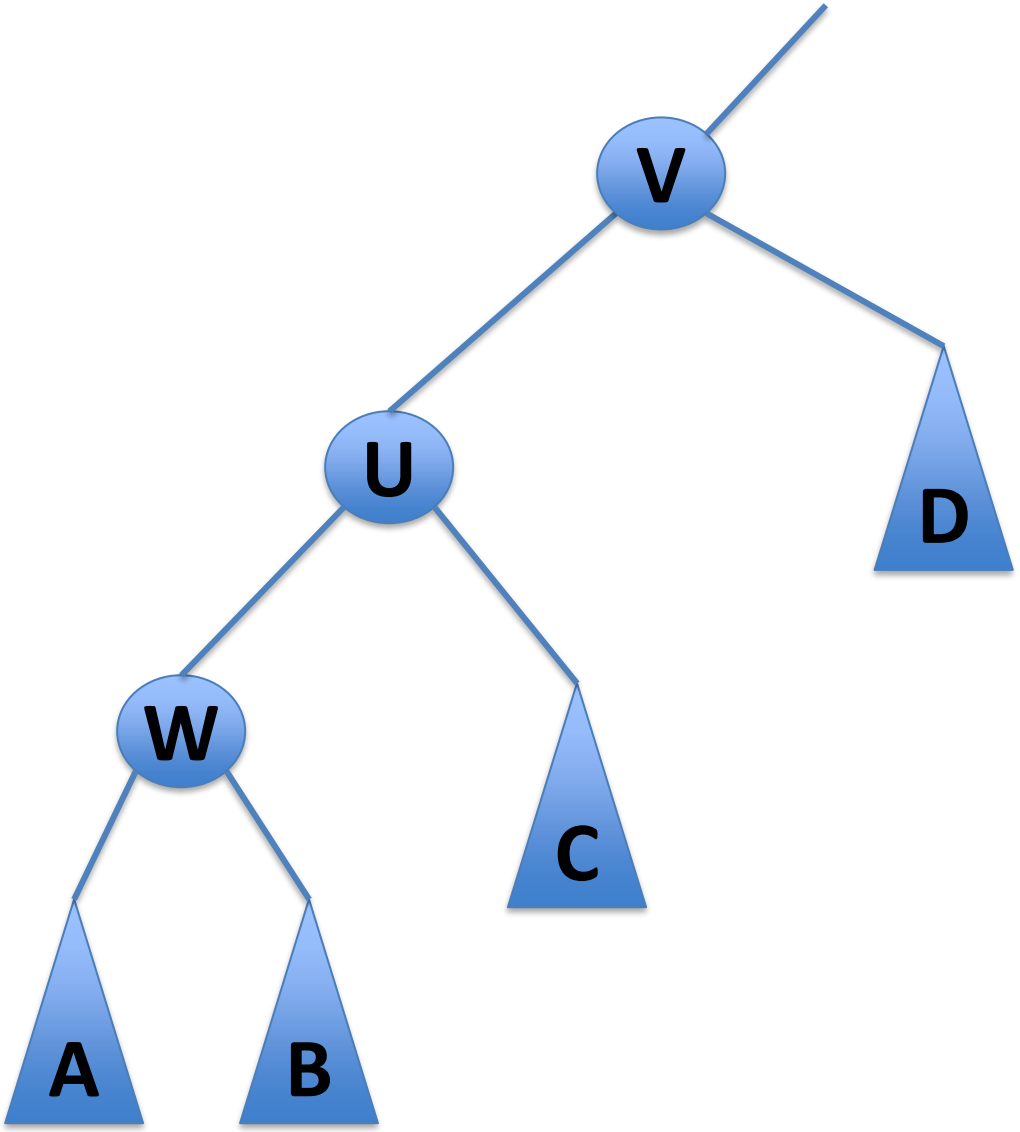
Theorem 7.2: A Treap implements the **SSet** interface. A Treap supports the operations add(x), remove(x) and find(x) in expected $O(\log n)$ time.

TREAP

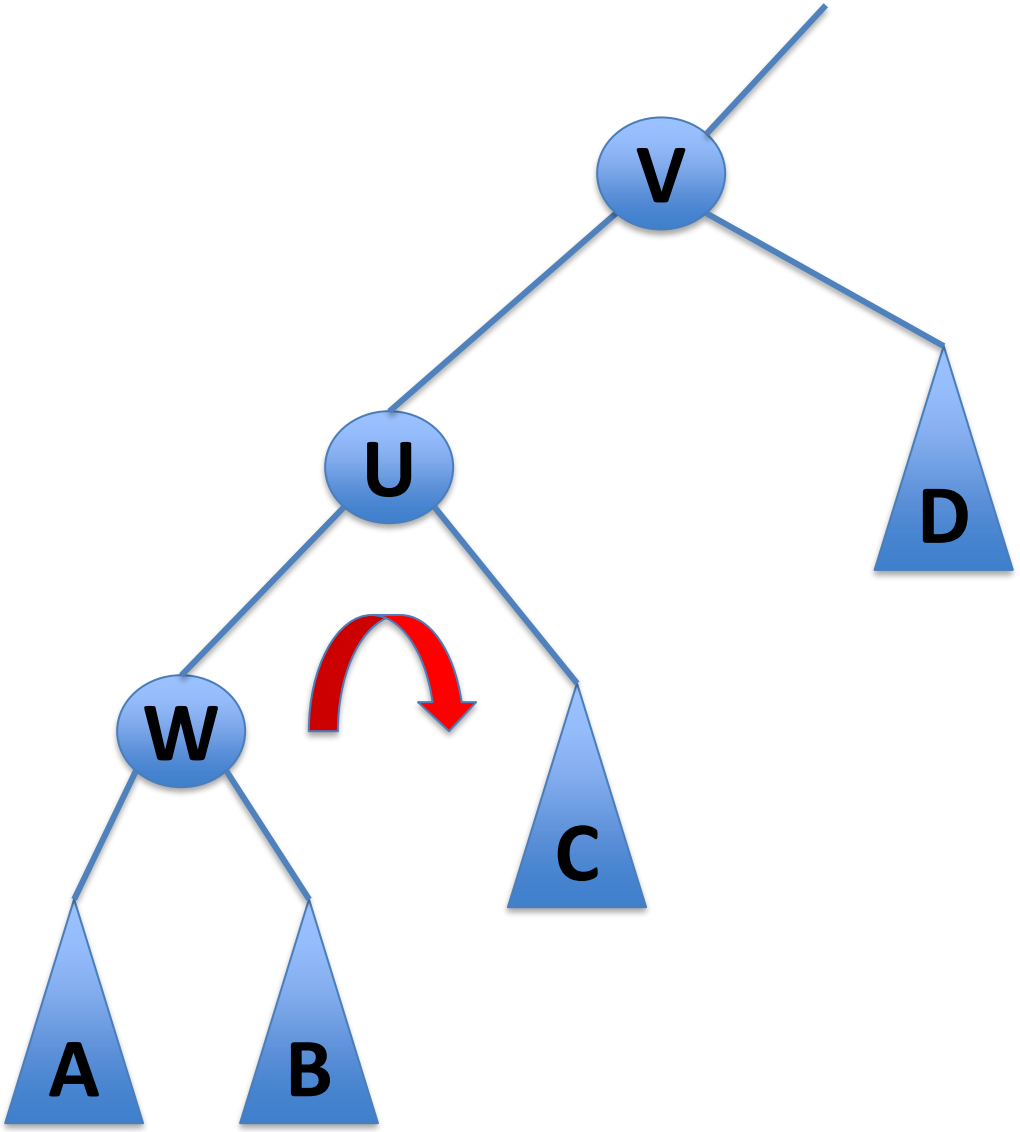
When there are multiple rotations, what happens to the height of the tree?

Note: It can be shown that the expected number of rotations is actually $O(1)$.

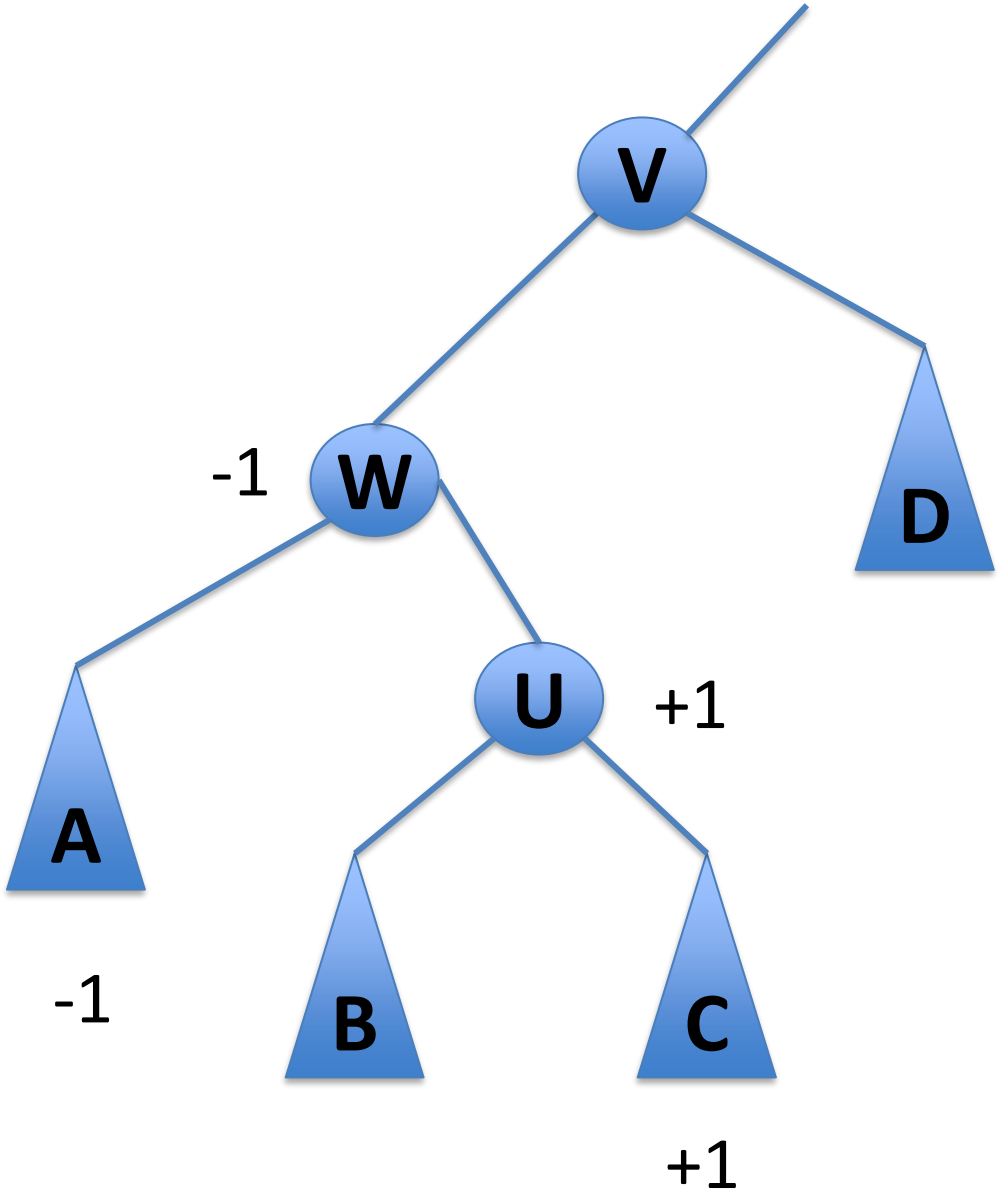
TREAP



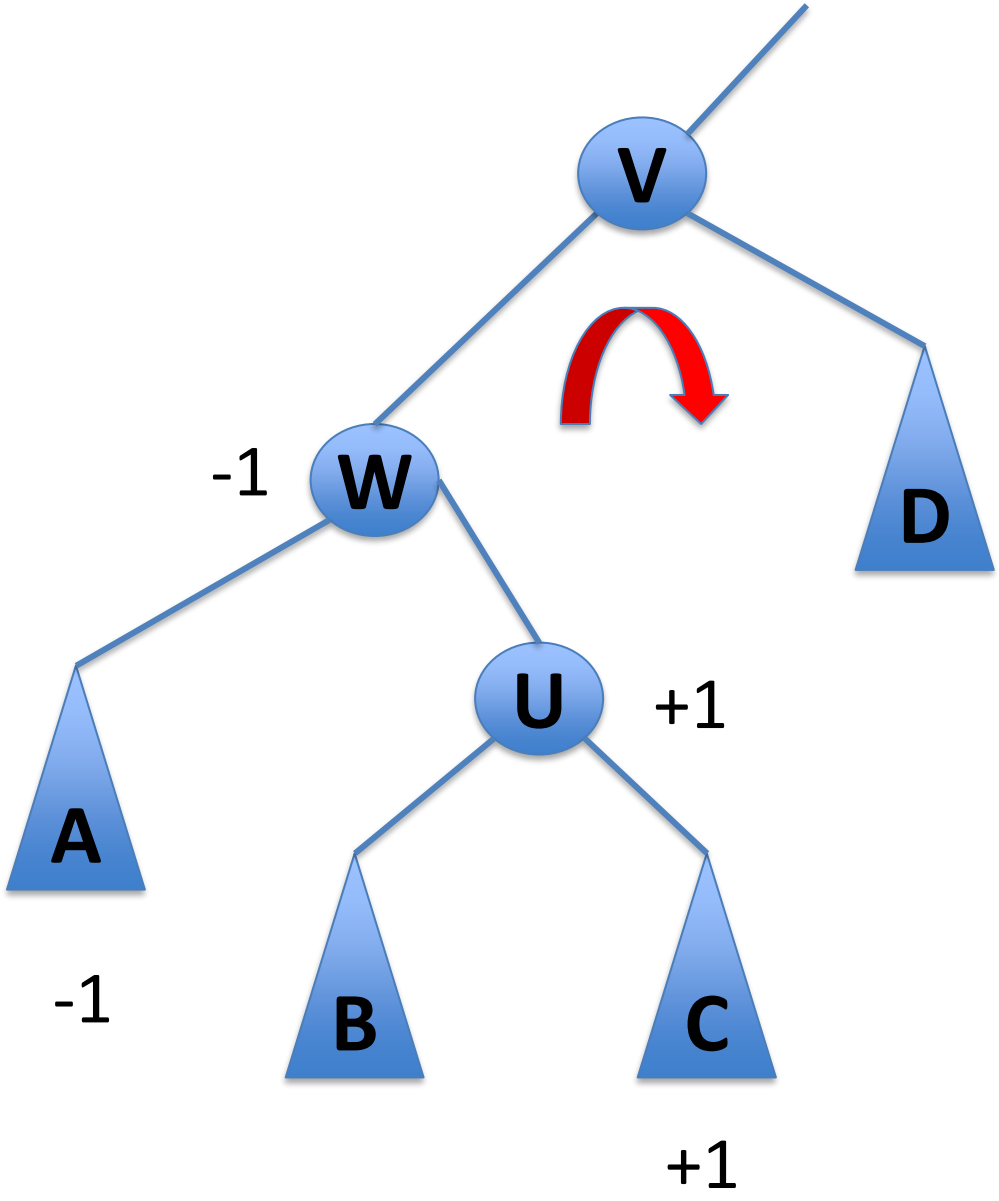
TREAP



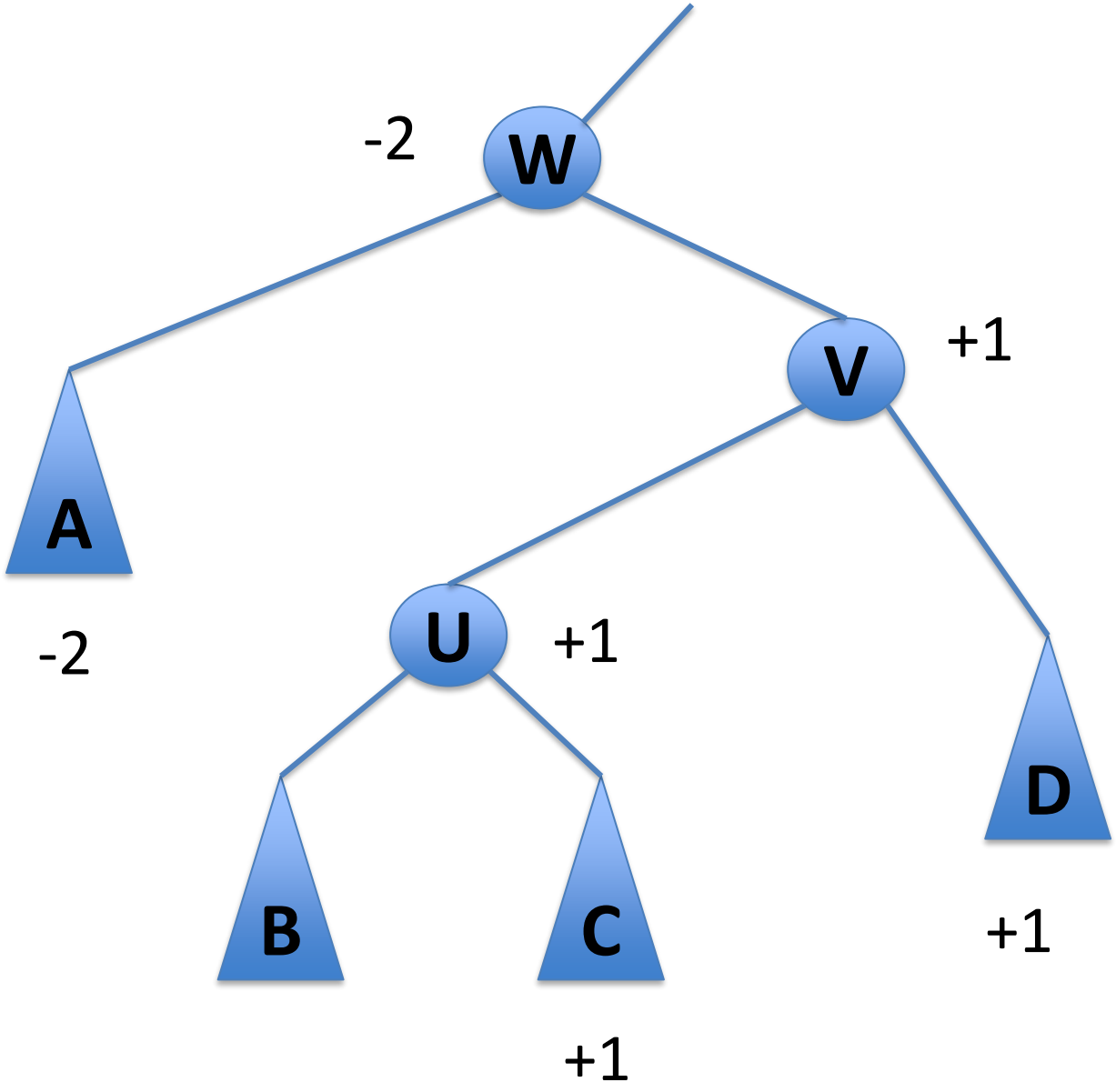
TREAP



TREAP



TREAP



Sorted Sets

	find, add, remove
Skip Lists	expected $O(\log n)$
BST	$O(height) \in O(n)$
Random BST	expected $O(\log n)$ [find]
Treap	expected $O(\log n)$